In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
df=pd.read_csv('bike_sharing.csv')
```

Exploratory data analysis

In [3]:
```
df.shape
```

Out[3]:
```
(10886, 12)
```

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   datetime    10886 non-null  object 
 1   season      10886 non-null  int64  
 2   holiday     10886 non-null  int64  
 3   workingday  10886 non-null  int64  
 4   weather     10886 non-null  int64  
 5   temp        10886 non-null  float64
 6   atemp       10886 non-null  float64
 7   humidity    10886 non-null  int64  
 8   windspeed   10886 non-null  float64
 9   casual      10886 non-null  int64  
 10  registered  10886 non-null  int64  
 11  count       10886 non-null  int64  
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
```

statistical summary of entire data frame

In [5]:
```
df.describe()
```

Out[5]:

	season	holiday	workingday	weather	temp	1C
count	10886.000000	10886.000000	10886.000000	10886.000000	10886.00000	
mean	2.506614	0.028569	0.680875	1.418427	20.23086	
std	1.116174	0.166599	0.466159	0.633839	7.79159	
min	1.000000	0.000000	0.000000	1.000000	0.82000	
25%	2.000000	0.000000	0.000000	1.000000	13.94000	
50%	3.000000	0.000000	1.000000	1.000000	20.50000	
75%	4.000000	0.000000	1.000000	2.000000	26.24000	
max	4.000000	1.000000	1.000000	4.000000	41.00000	

Null values

In [6]:
```
df.isnull().sum()
```

Out[6]:

	0
datetime	0
season	0
holiday	0
workingday	0
weather	0
temp	0
atemp	0
humidity	0
windspeed	0
casual	0
registered	0
count	0

dtype: int64

There are no null values in the dataset

Duplicate Values

In [7]:
```
df.duplicated().value_counts()
```

Out[7]:

	count
False	10886

dtype: int64

There are no duplicate records in the dataset

In [8]:
```
df
```

Out[8]:

	datetime	season	holiday	workingday	weather	temp	atemp	humidity
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75
...	...	...	...	...	...	...	...	...
10881	2012-12-19 19:00:00	4	0	1	1	15.58	19.695	50
10882	2012-12-19 20:00:00	4	0	1	1	14.76	17.425	57
10883	2012-12-19 21:00:00	4	0	1	1	13.94	15.910	61
10884	2012-12-19 22:00:00	4	0	1	1	13.94	17.425	61
10885	2012-12-19 23:00:00	4	0	1	1	13.12	16.665	66

10886 rows × 12 columns

Categorical columns

In [9]:
```
cat_col=[]
for col in df.columns:
  if (df[col].value_counts().size<=5):
    cat_col.append(col)
cat_col
```

Out[9]:
```
['season', 'holiday', 'workingday', 'weather']
```

Numerical columns

In [10]:
```
numeric_col=[]
for col in df.columns:
  if (df[col].value_counts().size>=5):
```

```
    numeric_col.append(col)
numeric_col
```

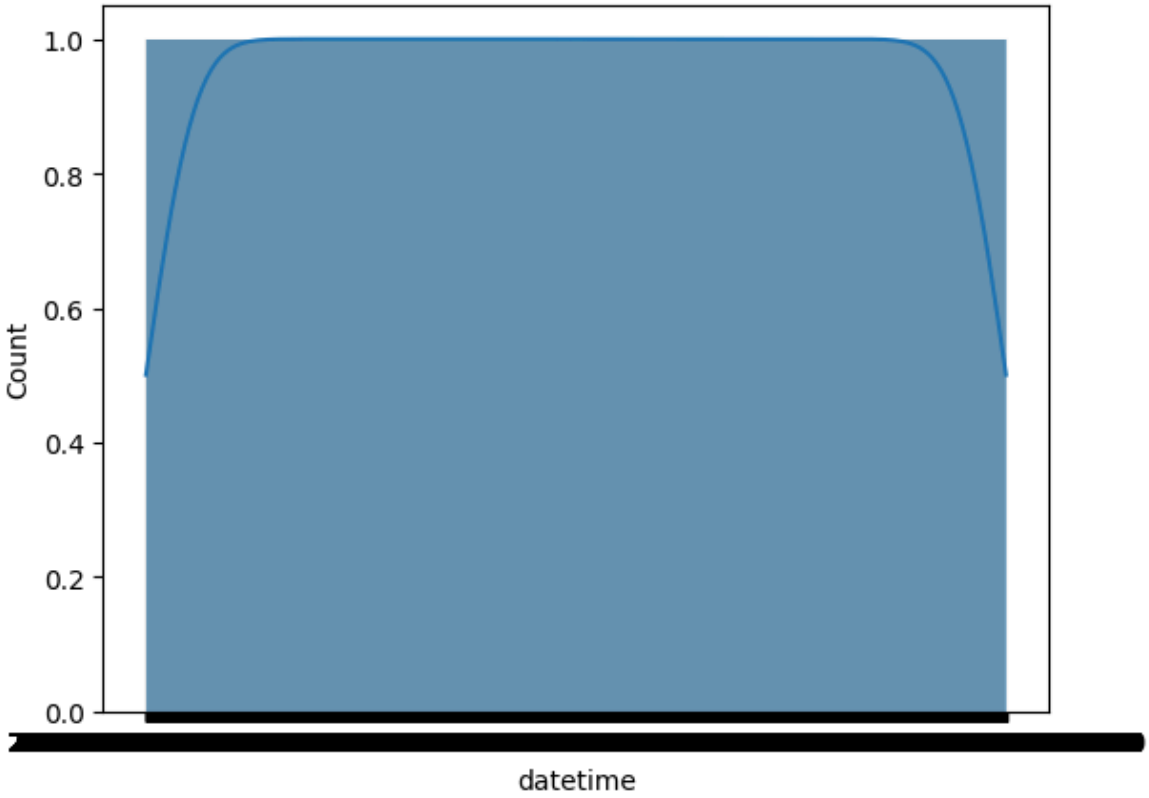
Out[10]:
```
['datetime',
 'temp',
 'atemp',
 'humidity',
 'windspeed',
 'casual',
 'registered',
 'count']
```

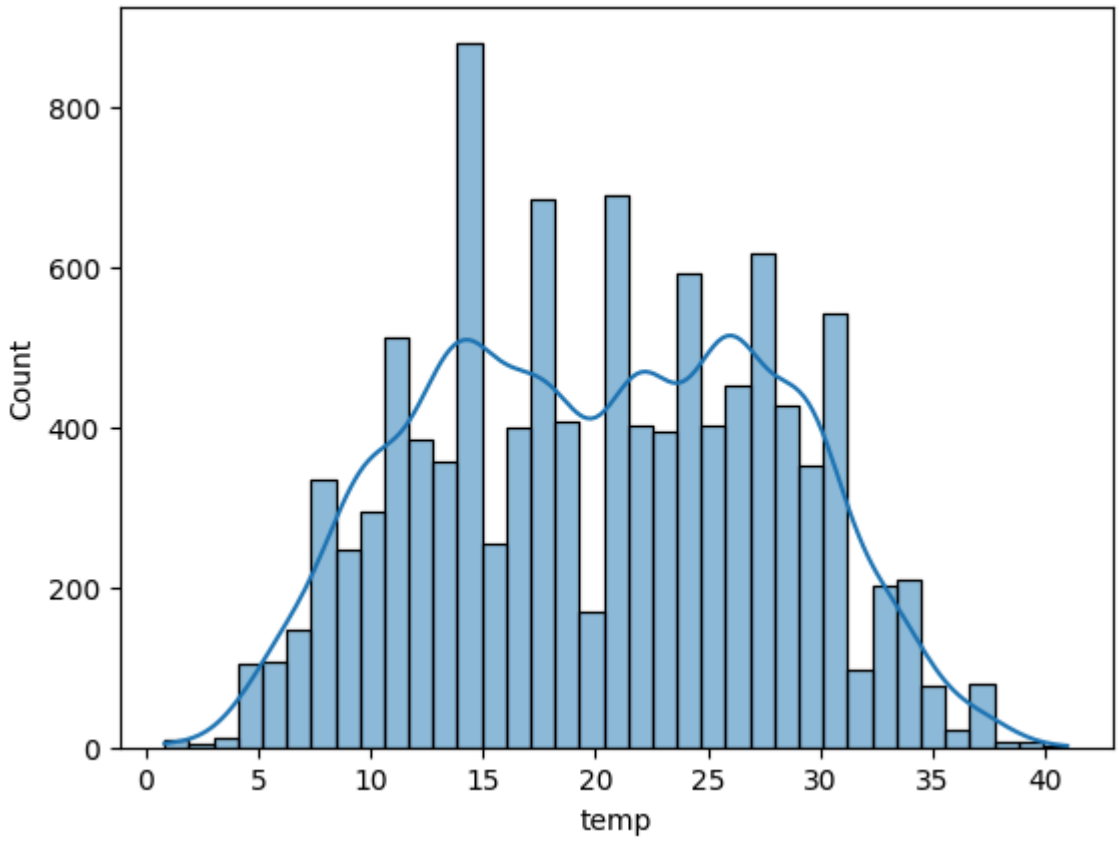
In [11]:
```
df['temp'].value_counts().size
```

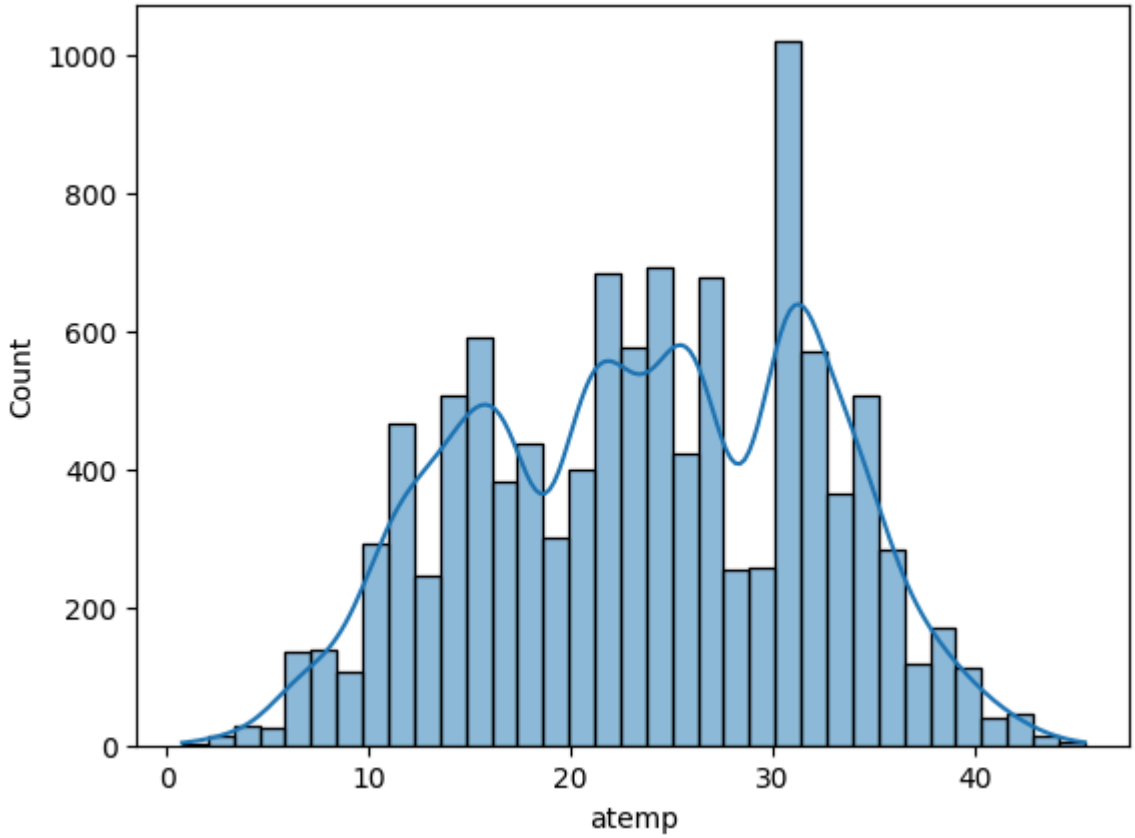
Out[11]: 49

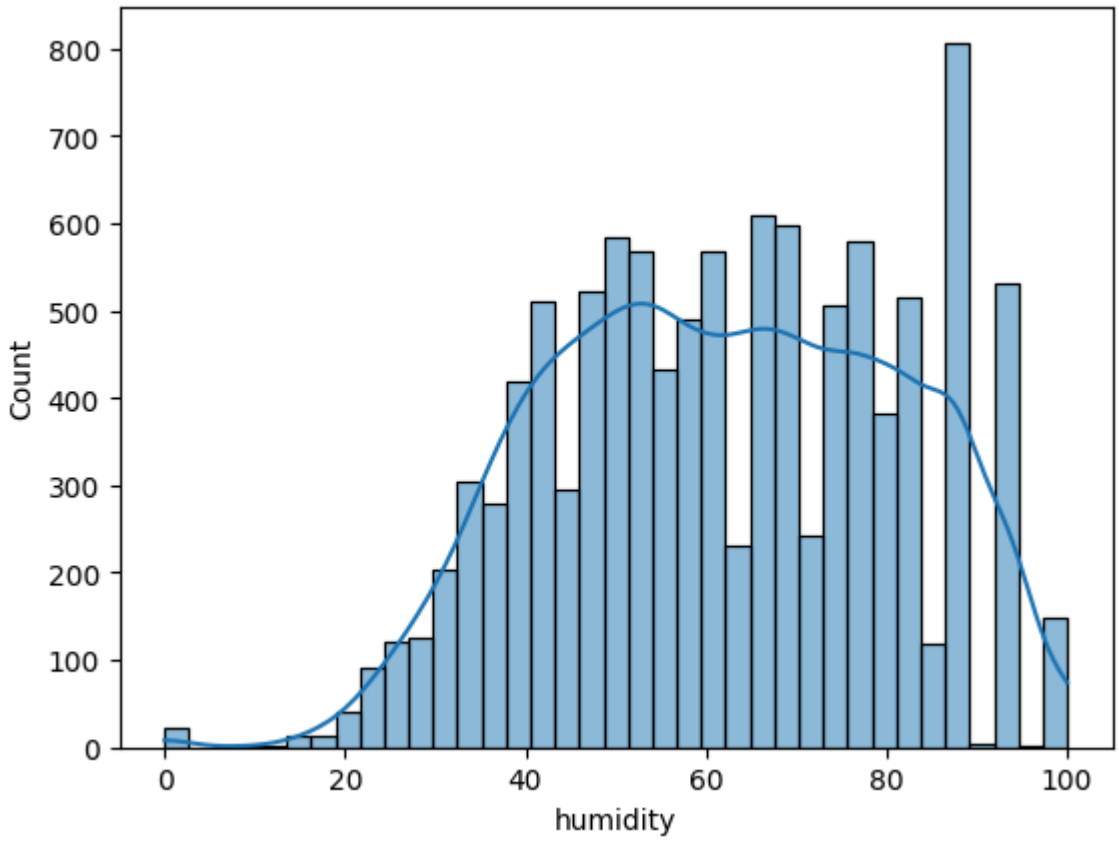
Univariate Analysis for numerical columns

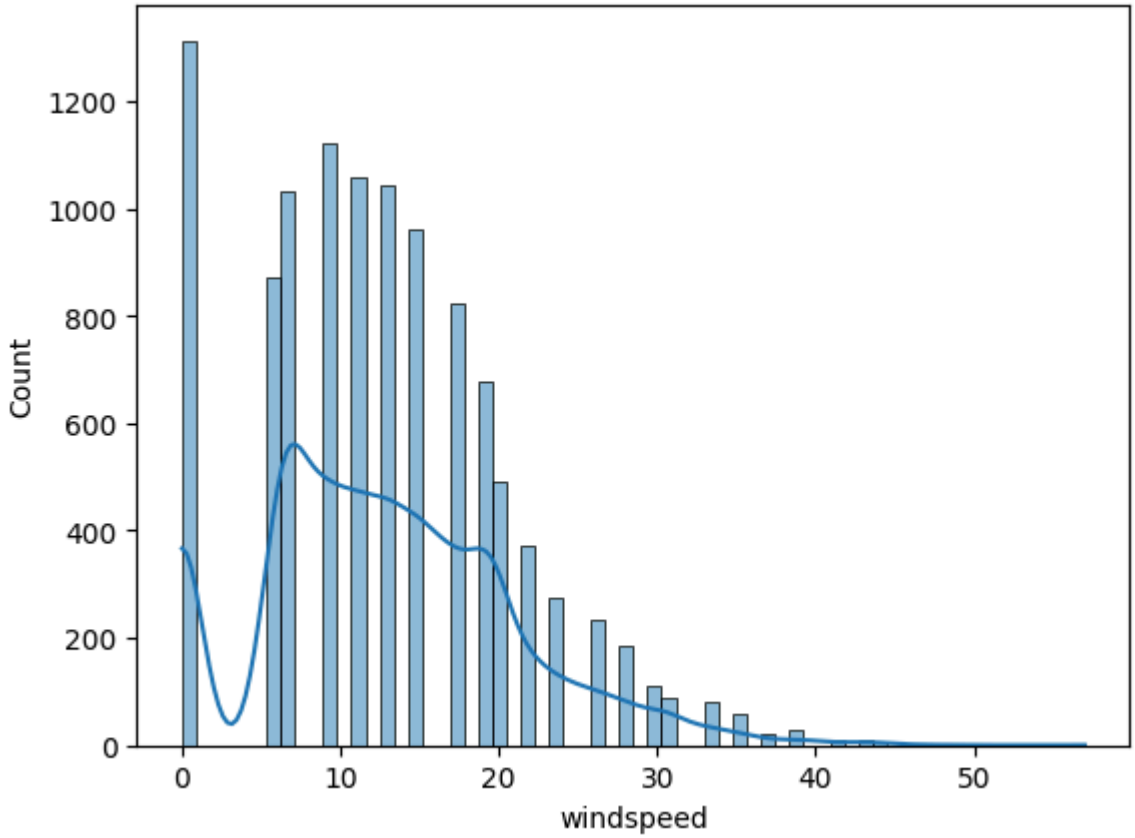
In [12]:
```
for col in numeric_col:
  sns.histplot(df[col],kde=True)
  plt.show()
```

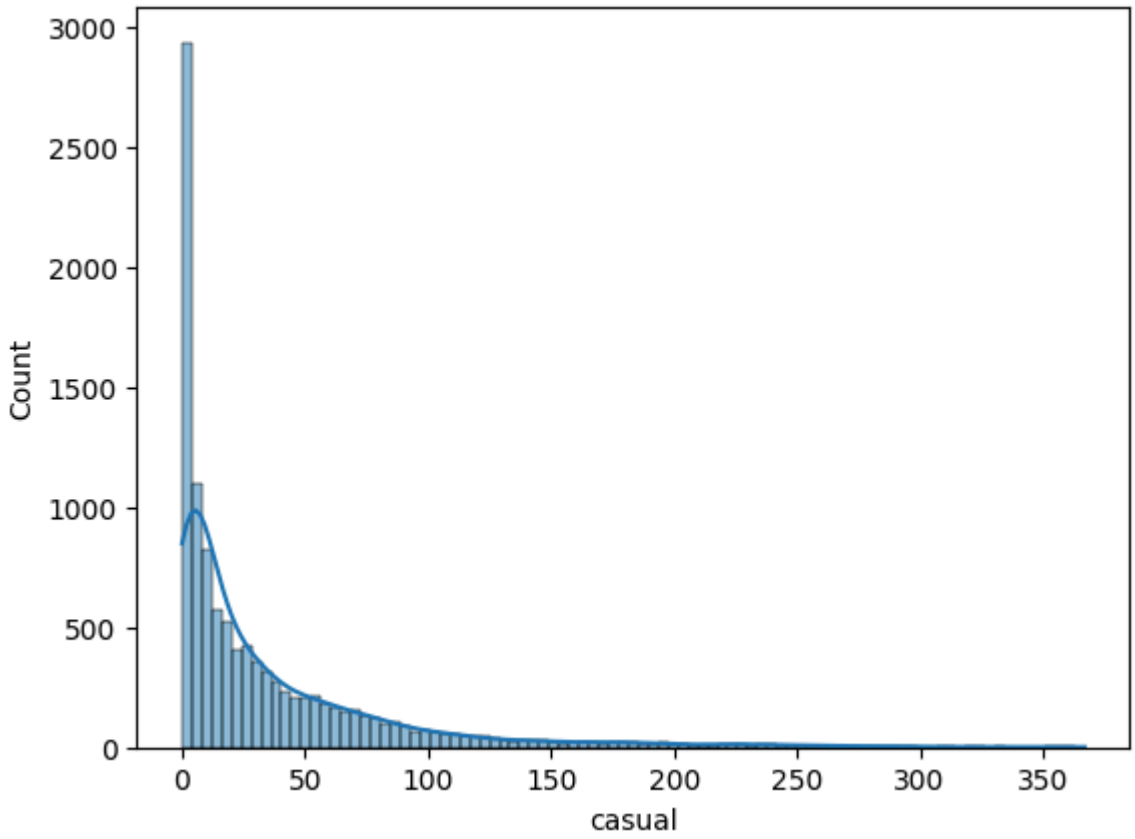


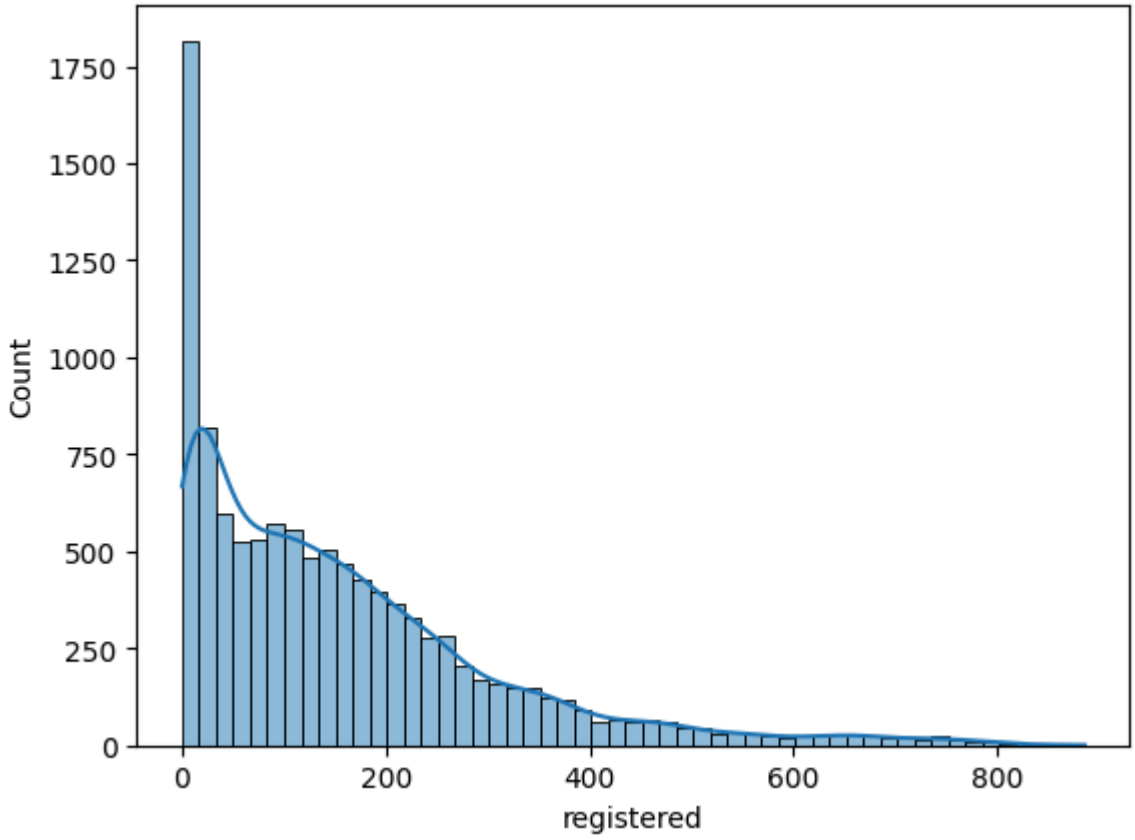

Count
800
600
400
200
0
0
5
10
15
20
25
30
35
40
temp

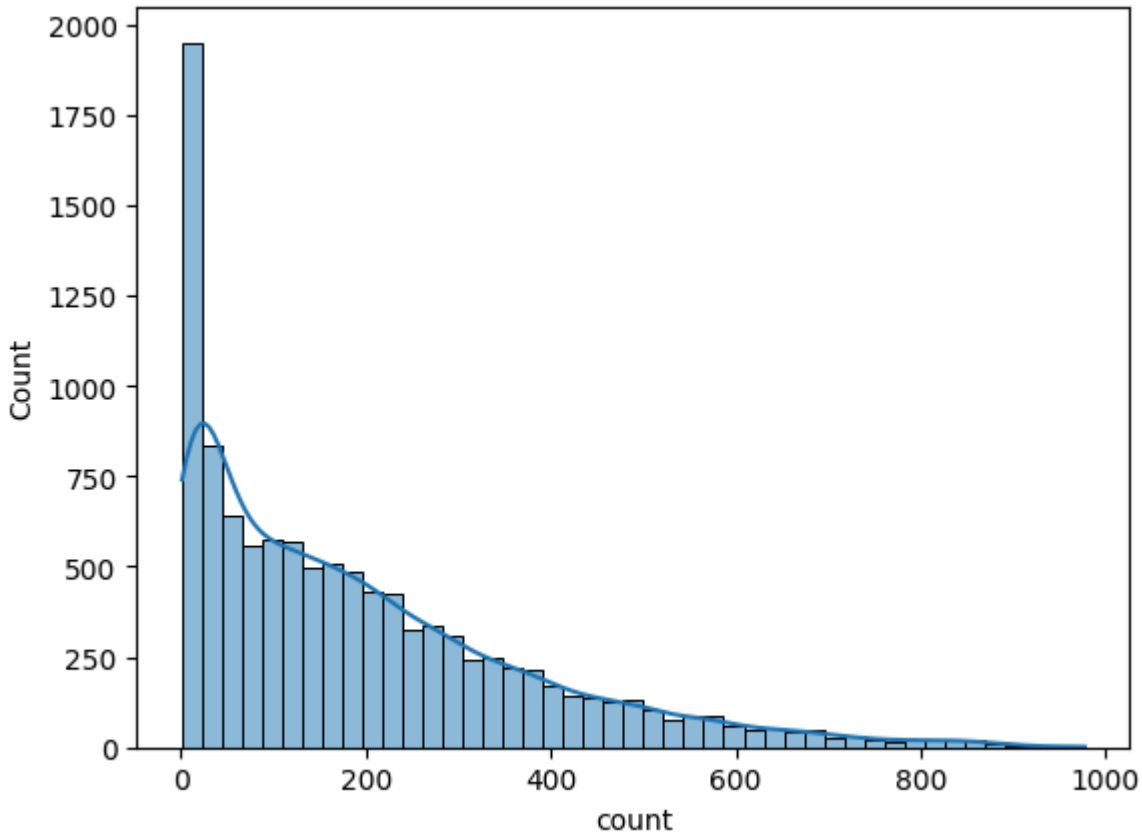

Count
1000
800
600
400
200
0
0
10
20
30
40
atemp


800
700
600
500
400
300
200
100
0
Count
0
20
40
60
80
100
humidity


1200
1000
800
600
400
200
0
Count
0
10
20
30
40
50
windspeed

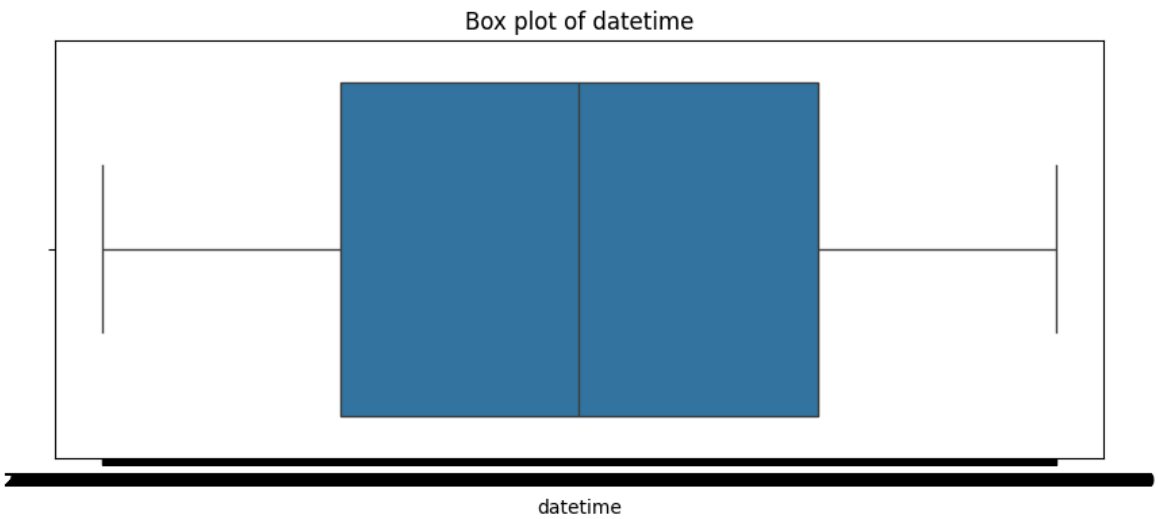

3000
2500
2000
1500
1000
500
0
Count
0
50
100
150
200
250
300
350
casual

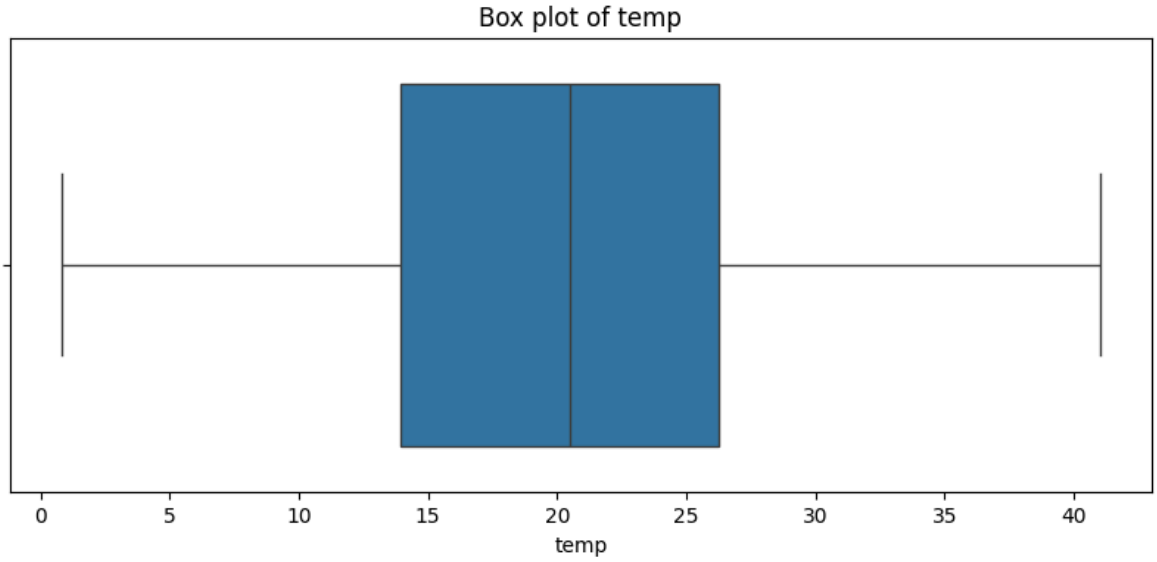

1750
1500
1250
1000
750
500
250
0
Count
0
200
400
600
800
registered

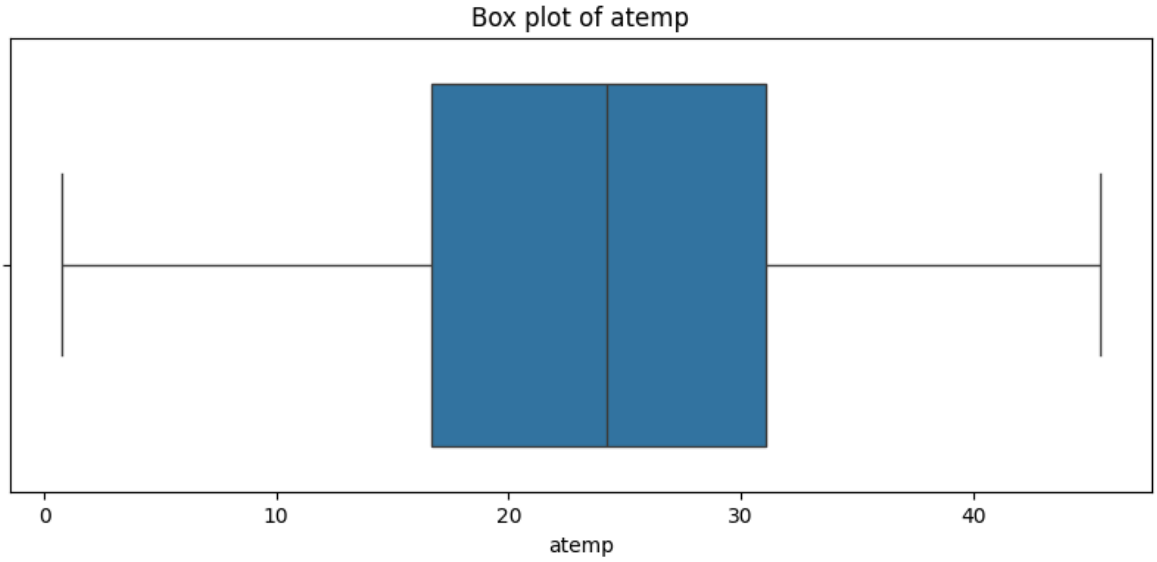


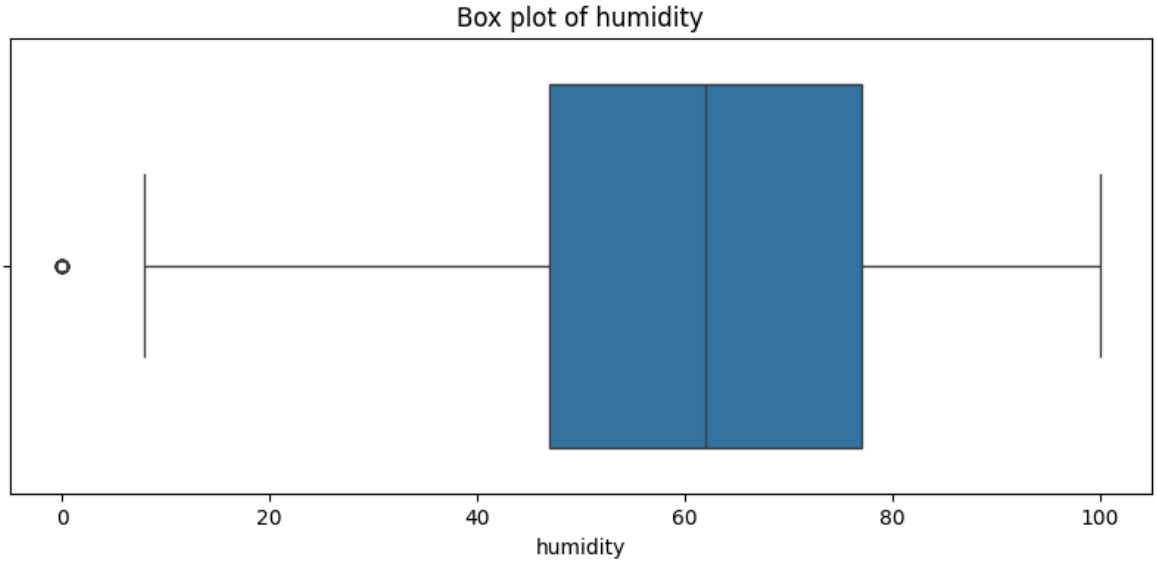
Box plot to identify outliers in numerical data

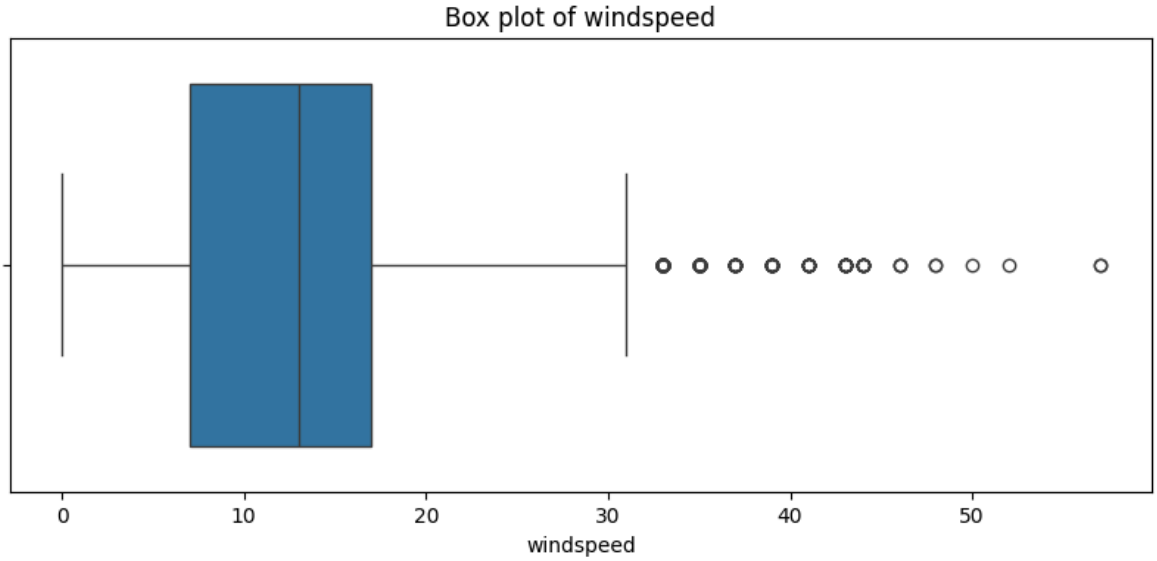
In [13]:
```
for col in numeric_col:
    plt.figure(figsize=(10, 4))
    sns.boxplot(x=df[col])
    plt.title(f'Box plot of {col}')
    plt.show()
```

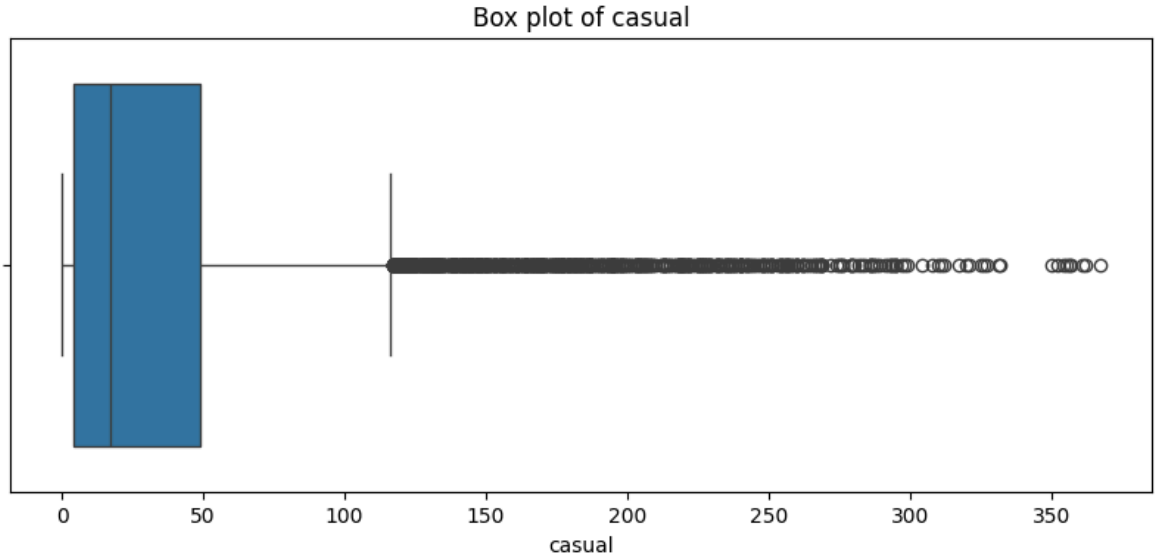


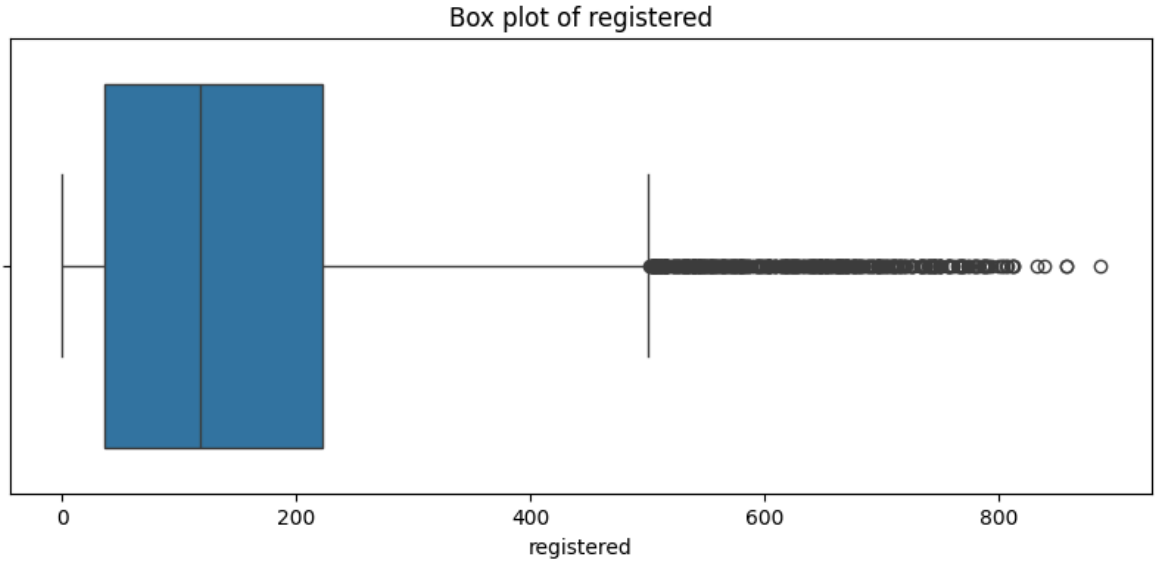

Box plot of temp
0
5
10
15
20
25
30
35
40
temp

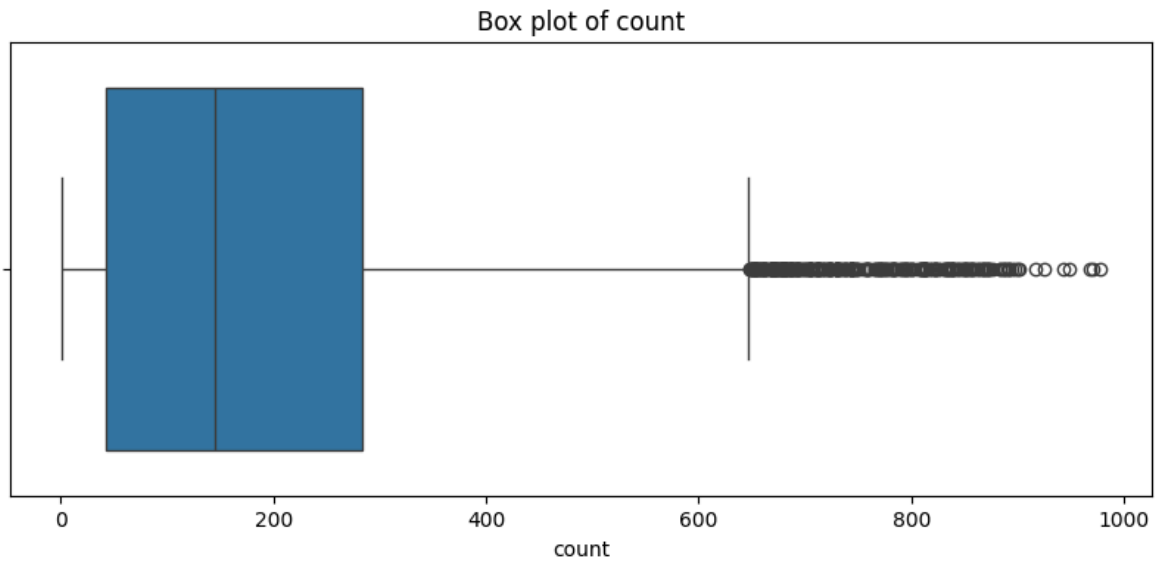

Box plot of atemp
0
10
20
30
40
atemp


Box plot of humidity
0
20
40
60
80
100
humidity


Box plot of windspeed
0
10
20
30
40
50
windspeed

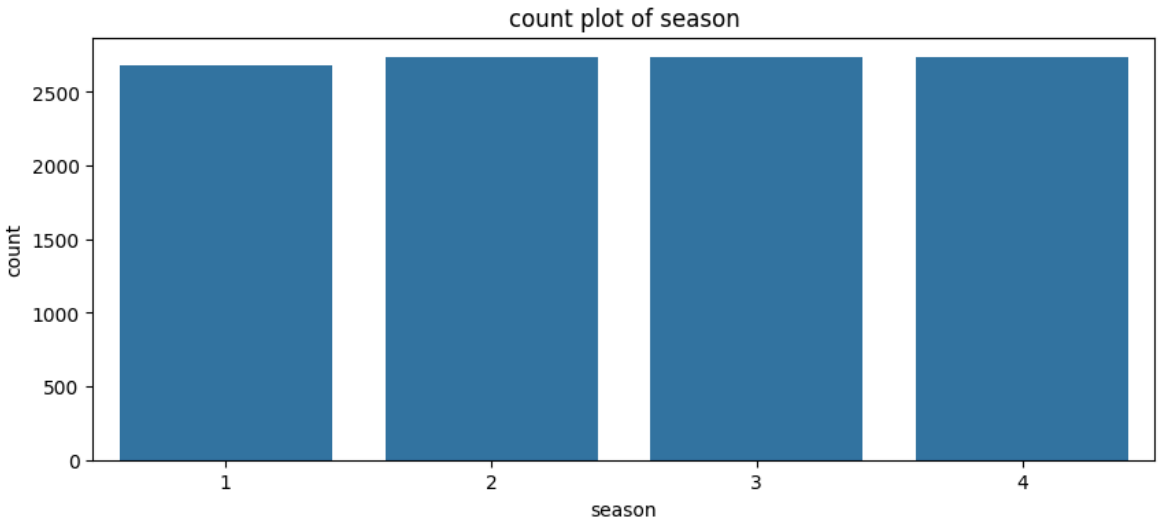

Box plot of casual
0
50
100
150
200
250
300
350
casual

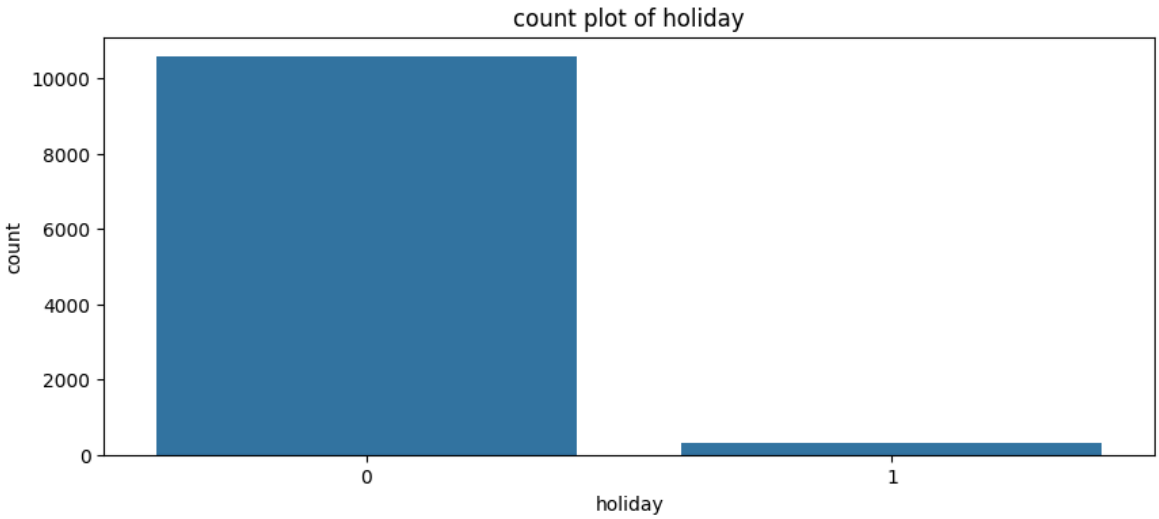

Box plot of registered
0
200
400
600
800
registered

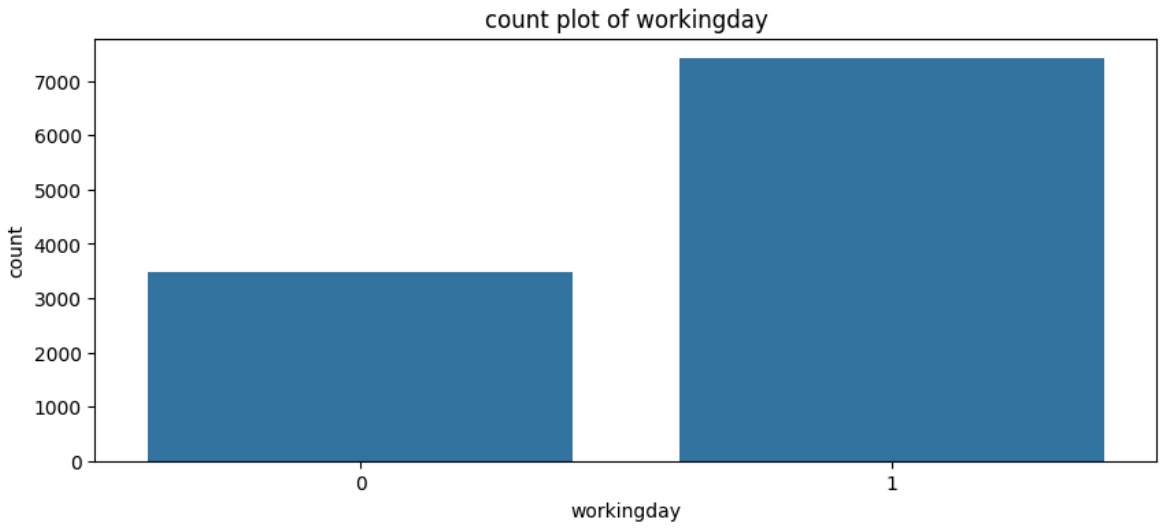


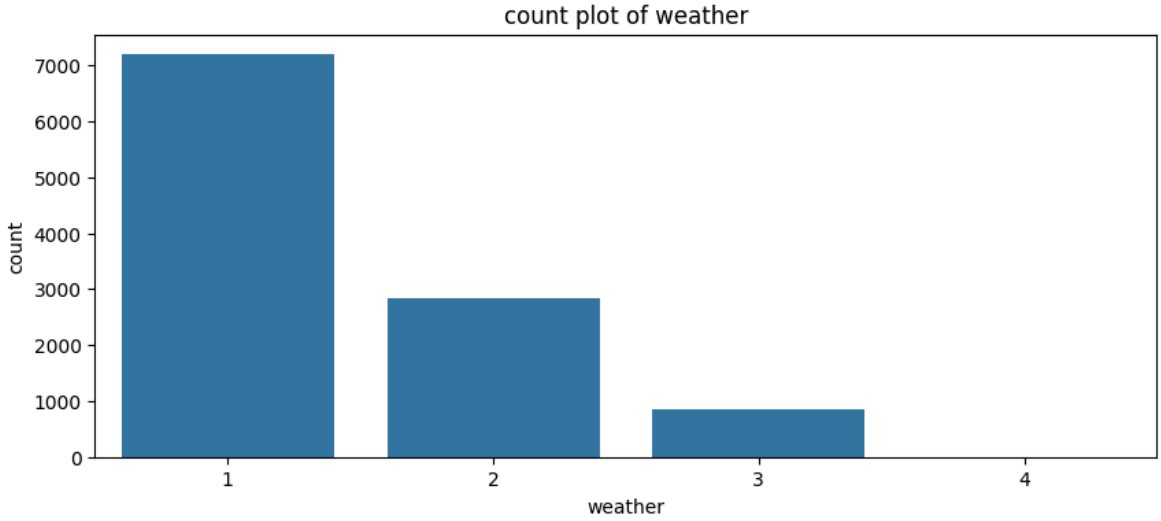
univariate analysis for categorical columns

In [14]:
```
for col in cat_col:
    plt.figure(figsize=(10, 4))
    sns.countplot(x=df[col])
    plt.title(f'count plot of {col}')
    plt.show()
```






count plot of workingday
count
7000
6000
5000
4000
3000
2000
1000
0
0
1
workingday


count plot of weather
count
7000
6000
5000
4000
3000
2000
1000
0
1
2
3
4
weather

In [15]:
```
df
```

Out[15]:

	datetime	season	holiday	workingday	weather	temp	atemp	humidity
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75
...	...	...	...	...	...	...	...	...
10881	2012-12-19 19:00:00	4	0	1	1	15.58	19.695	50
10882	2012-12-19 20:00:00	4	0	1	1	14.76	17.425	57
10883	2012-12-19 21:00:00	4	0	1	1	13.94	15.910	61
10884	2012-12-19 22:00:00	4	0	1	1	13.94	17.425	61
10885	2012-12-19 23:00:00	4	0	1	1	13.12	16.665	66

10886 rows × 12 columns

Relationship

In [16]:
```
df_copy=df.copy()
```

In [17]:
```
df_copy=df_copy.drop('datetime',axis=1)
```

In [18]:
```
df_copy.corr()
```

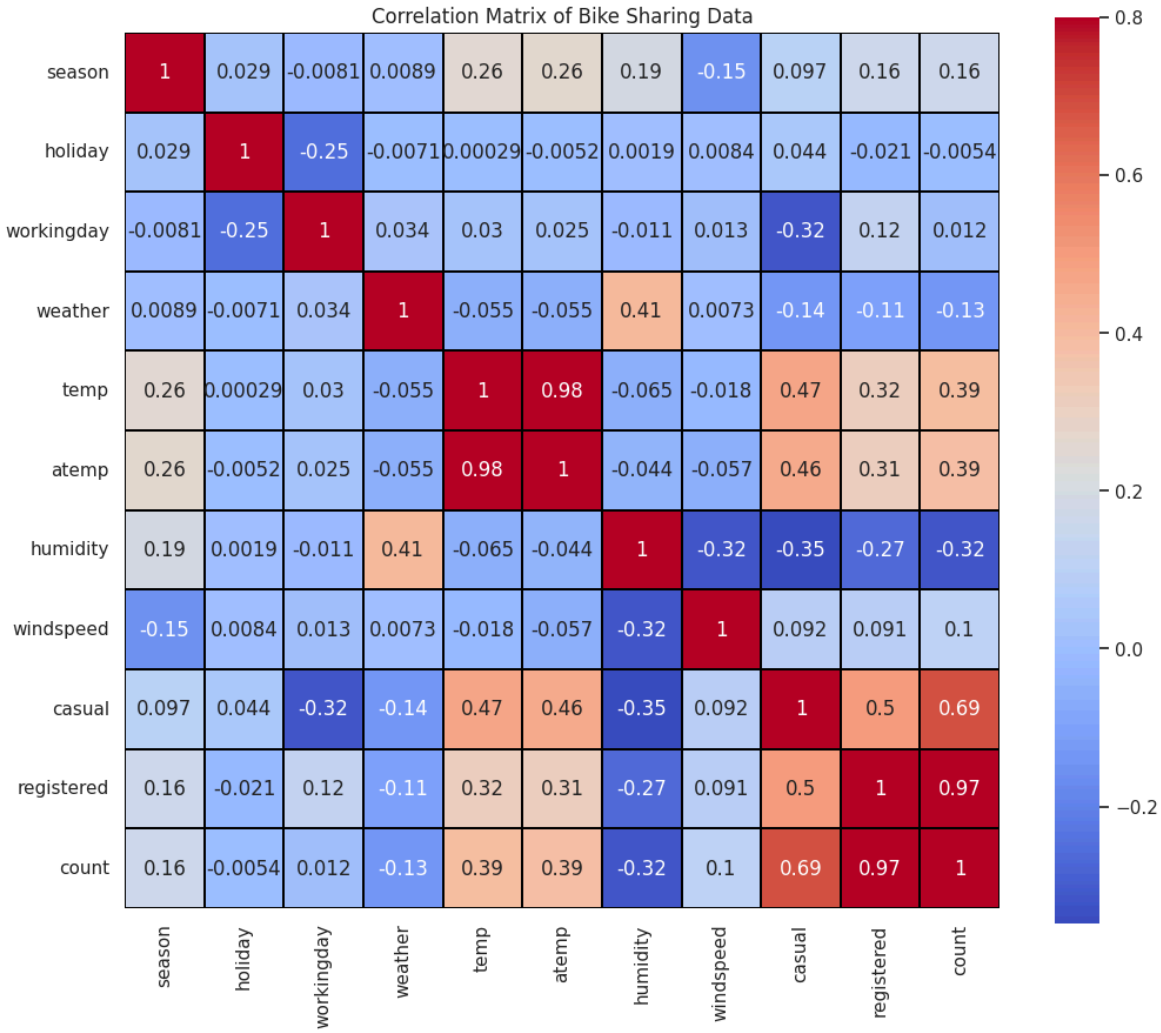
Out[18]:

	season	holiday	workingday	weather	temp	atemp
season	1.000000	0.029368	-0.008126	0.008879	0.258689	0.264744
holiday	0.029368	1.000000	-0.250491	-0.007074	0.000295	-0.005215
workingday	-0.008126	-0.250491	1.000000	0.033772	0.029966	0.024660
weather	0.008879	-0.007074	0.033772	1.000000	-0.055035	-0.055376
temp	0.258689	0.000295	0.029966	-0.055035	1.000000	0.984948
atemp	0.264744	-0.005215	0.024660	-0.055376	0.984948	1.000000
humidity	0.190610	0.001929	-0.010880	0.406244	-0.064949	-0.043536
windspeed	-0.147121	0.008409	0.013373	0.007261	-0.017852	-0.057473
casual	0.096758	0.043799	-0.319111	-0.135918	0.467097	0.462067
registered	0.164011	-0.020956	0.119460	-0.109340	0.318571	0.314635
count	0.163439	-0.005393	0.011594	-0.128655	0.394454	0.389784

Heat Map for representing relationship between columns

In [19]:

```
sns.set(font_scale=1.0)
plt.figure(figsize=(12,10))
correlation_values = df_copy.corr(method = 'pearson')
sns.heatmap(correlation_values, vmax = .8, linewidths=0.01, square=True,
plt.title('Correlation Matrix of Bike Sharing Data');
plt.show()
```



Insights (from run)

temp and atemp correlation ≈ 0.98 → essentially duplicate info.

registered and count correlation ≈ 0.97 (since count = casual + registered).

casual and count also strongly positively correlated (as expected).

Recommendation

Drop one of temp/atemp to avoid multicollinearity in linear models.

If predicting count, do not include registered and casual as predictors (they are components of the target); choose either predict count or model registered/casual separately.

In [20]:
```
# df = df.drop(['registered', 'atemp'], axis=1)
# display(df.head())
```

Checking if there any significant difference between the no. of bike rides on Weekdays and Weekends

Hypothesis

H0(Null Hypothesis):there is no significant difference between the no. of bike rides on Weekdays and Weekends

H1(Alternate Hypothesis):there is significant difference between the no. of bike rides on Weekdays and Weekends

In [21]:
```
weekdays=df[df['workingday']==1]['count']
weekdays
```

Out[21]:

	count
47	5
48	2
49	1
50	3
51	30
...	...
10881	336
10882	241
10883	168
10884	129
10885	88

7412 rows × 1 columns

dtype: int64

In [22]:
```
weekends=df[(df['workingday']==0) & (df['holiday']==0)]['count']
weekends
```

Out[22]:

	count
0	16
1	40
2	32
3	13
4	1
...	...
10809	109
10810	122
10811	106
10812	89
10813	33

3163 rows × 1 columns

dtype: int64

In [23]:
```
from scipy.stats import ttest_ind
```

In [ ]:
```
##Test
```

In [24]:
```
ttest_ind(weekdays,weekends)
```

Out[24]:
```
TtestResult(statistic=np.float64(1.101622575515633), pvalue=np.float64
(0.27065086640542324), df=np.float64(10573.0))
```

Results

Interpretation of Results:
The calculated p-value from the t-test is 0.27065.
The significance level (alpha) is 0.05.

Since the p-value (0.27065) is greater than the significance level (0.05), we fail to reject the Null Hypothesis.

Decision

Conclusion: Based on the t-test results, there is no statistically significant difference in the average number of bike rides between weekdays and weekends (non-holiday days) at a 0.05 significance level.

Recommendation

Consider segmenting by season/weather; workingday effect may interact with season.

In [24]:

Check if the demand of bicycles on rent is the same for different Weather

conditions?

Hypothesis

H0(Null Hypothesis):The demand of bicycles on rent is the same for different Weather conditions

H1(Alternate Hypothesis):The demand of bicycles on rent is not same for different Weather conditions

In [25]:
```
df['weather'].value_counts()
```

Out[25]:

weather	count
1	7192
2	2834
3	859
4	1

dtype: int64

In [26]:
```
df[['weather','count']]
```

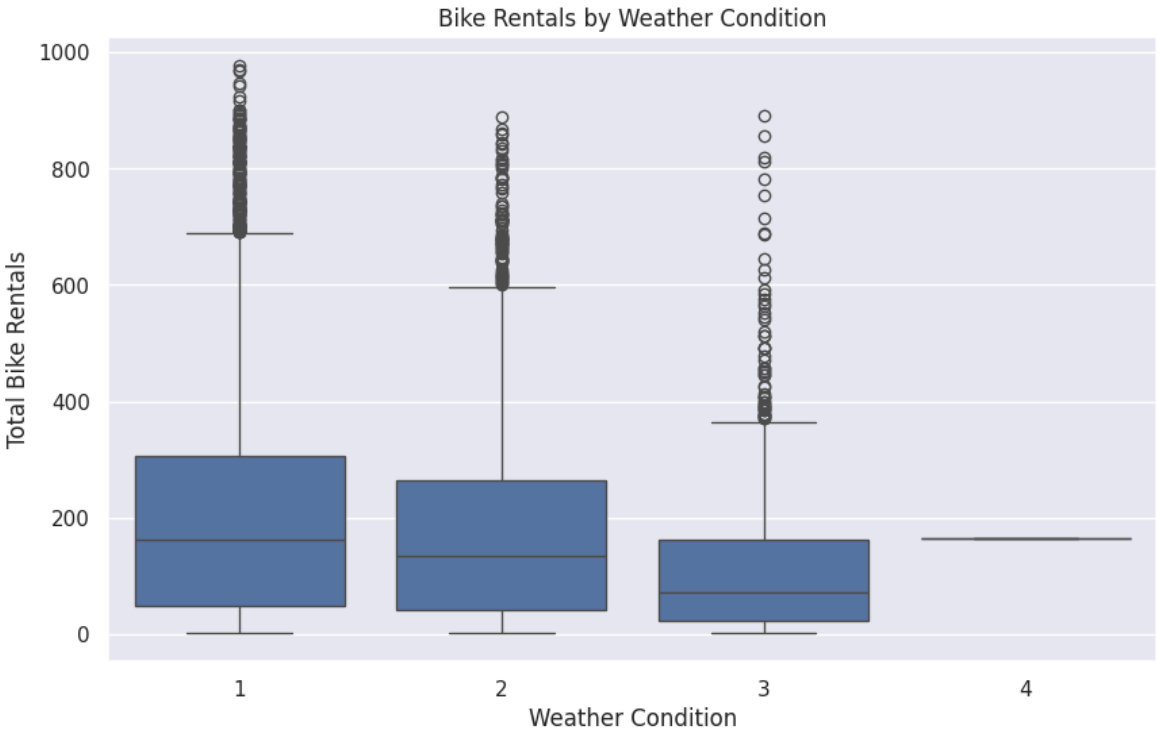
Out[26]:

	weather	count
0	1	16
1	1	40
2	1	32
3	1	13
4	1	1
...	...	...
10881	1	336
10882	1	241
10883	1	168
10884	1	129
10885	1	88

10886 rows × 2 columns

First, let's observe the distribution of bike rentals across different weather conditions using a box plot. This will give us a visual idea of whether there are noticeable differences before performing the statistical test.

In [27]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(x='weather', y='count', data=df)
plt.title('Bike Rentals by Weather Condition')
plt.xlabel('Weather Condition')
plt.ylabel('Total Bike Rentals')
plt.show()
```



Test

When we have one categorical variable with more than two groups and one continuous variable we consider one way anova test if assumptions are met and if not met we perform kruskal wallis test

Assumptions of anova

1. Normality of data we use shapiro wilk test for checking normality of data

In [28]:
```
df['weather'].value_counts()
```

Out[28]:

weather	count
1	7192
2	2834
3	859
4	1

dtype: int64

In [29]:
```
dis1=df[df['weather']==1]['count']
```

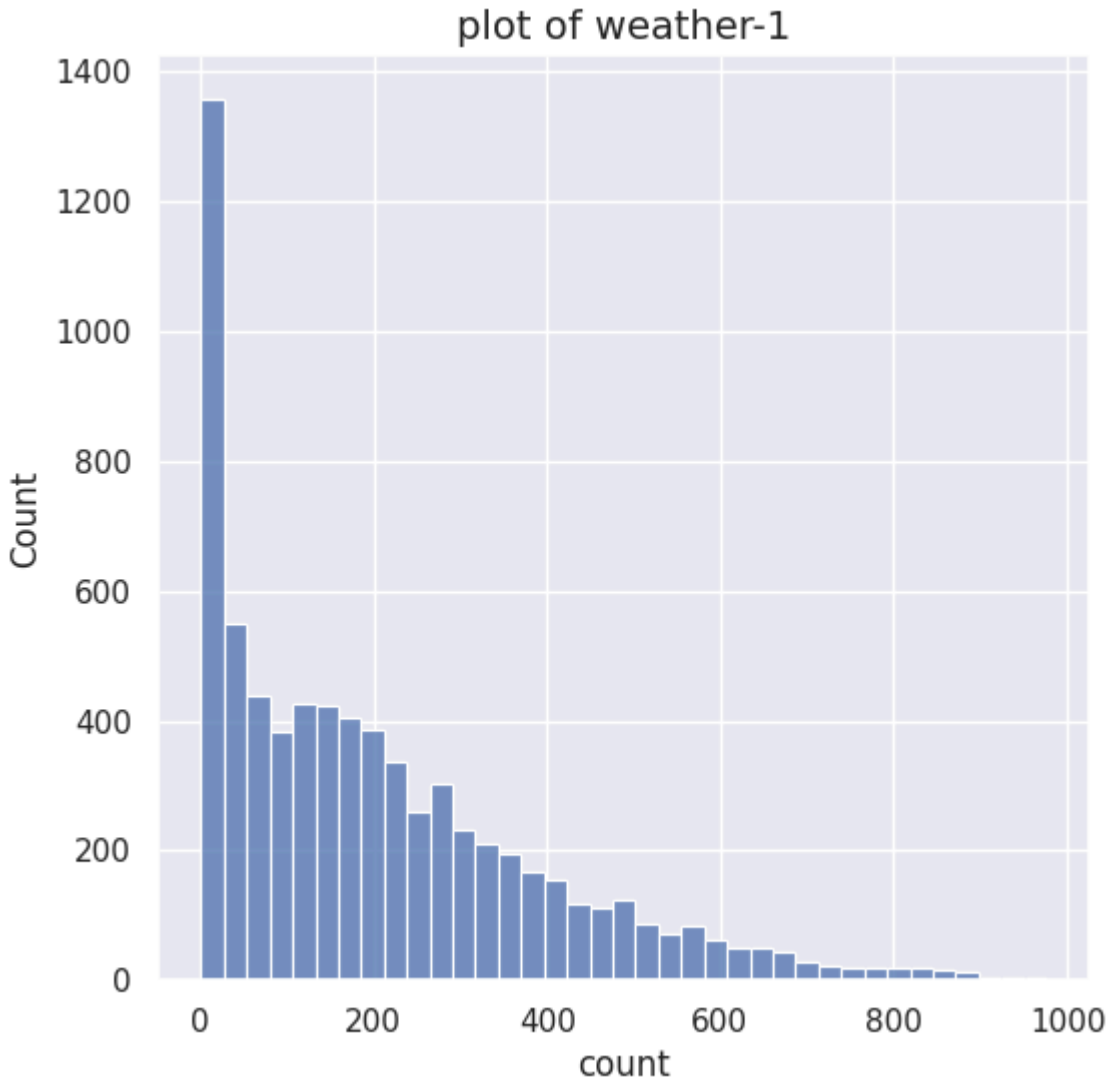
In [30]:
```
dis2=df[df['weather']==2]['count']
```

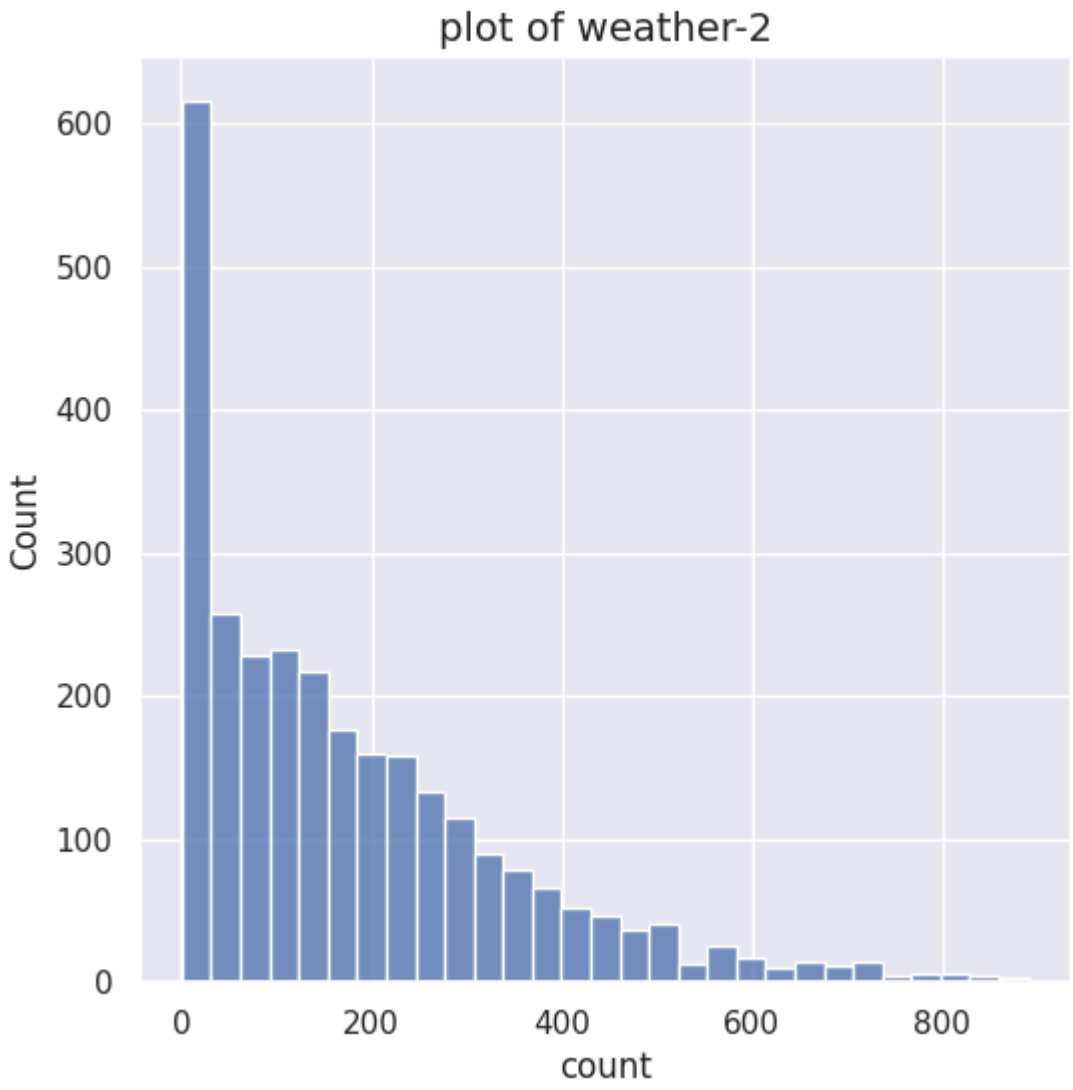
In [31]:
```
dis3=df[df['weather']==3]['count']
```

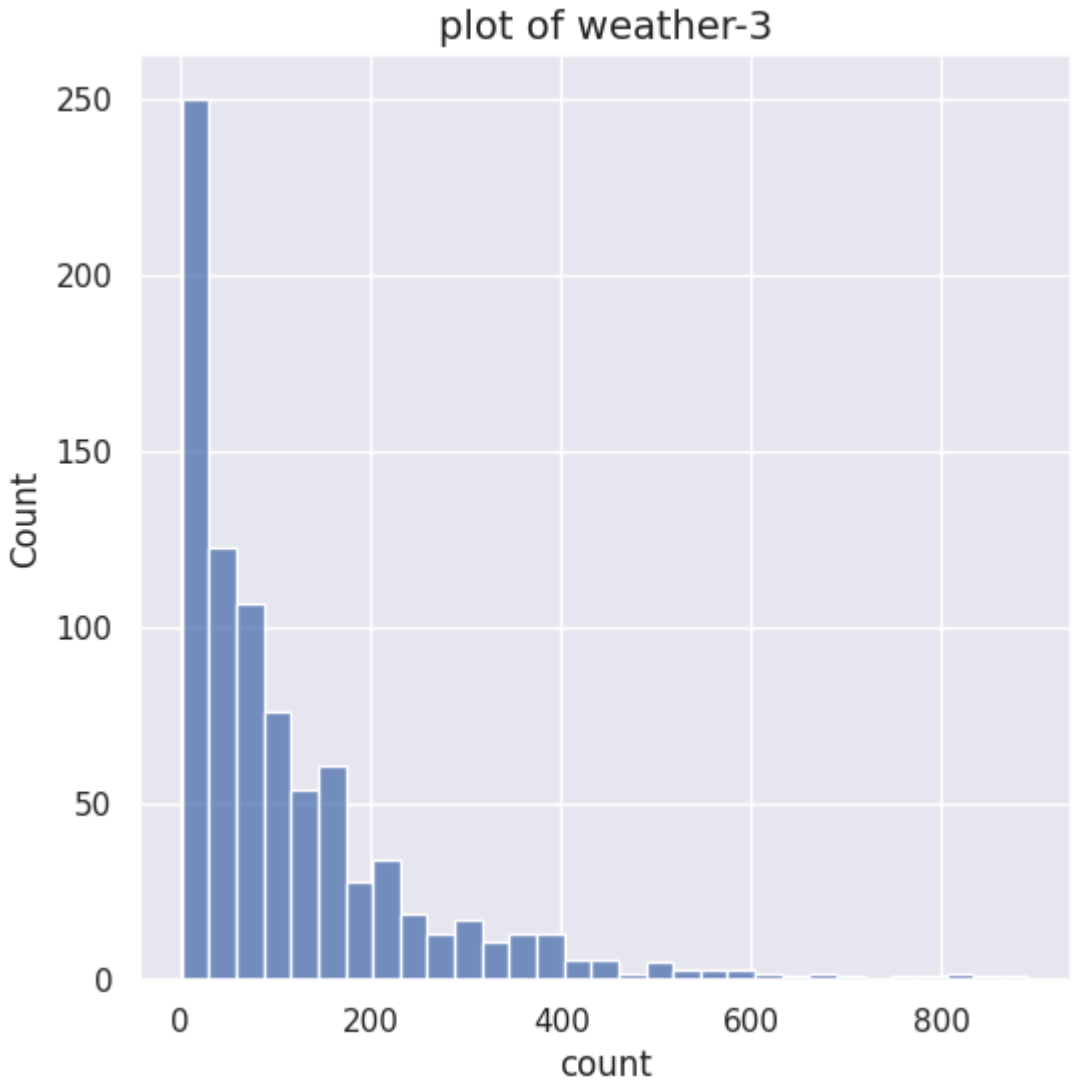
In [32]:
```
dis4=df[df['weather']==4]['count']
```

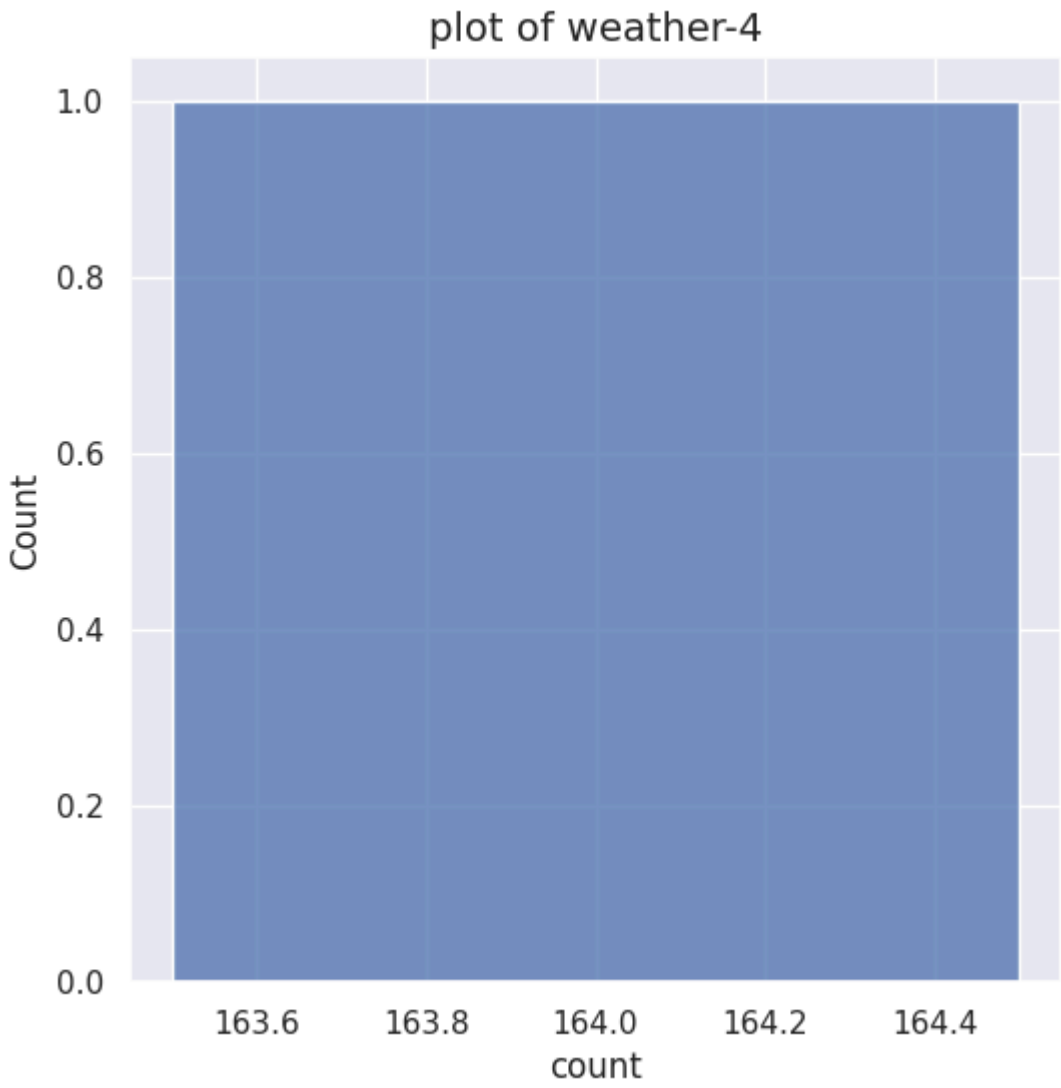
In [33]:
```
distributions=[dis1,dis2,dis3,dis4]
```

In [34]:
```
for col in distributions:
  # sns.histplot(col)
  # plt.show()
  distributions = {
    'weather-1': dis1,
    'weather-2': dis2,
    'weather-3': dis3,
    'weather-4': dis4
}

for col_name, series in distributions.items():
    plt.figure(figsize=(6,6))
    plt.title(f"plot of {col_name}", fontsize=14)
    sns.histplot(series)

    plt.show()
```


plot of weather-1
Count
count
0
200
400
600
800
1000
1200
1400
0
200
400
600
800
1000


plot of weather-2
Count
600
500
400
300
200
100
0
0
200
400
600
800
count

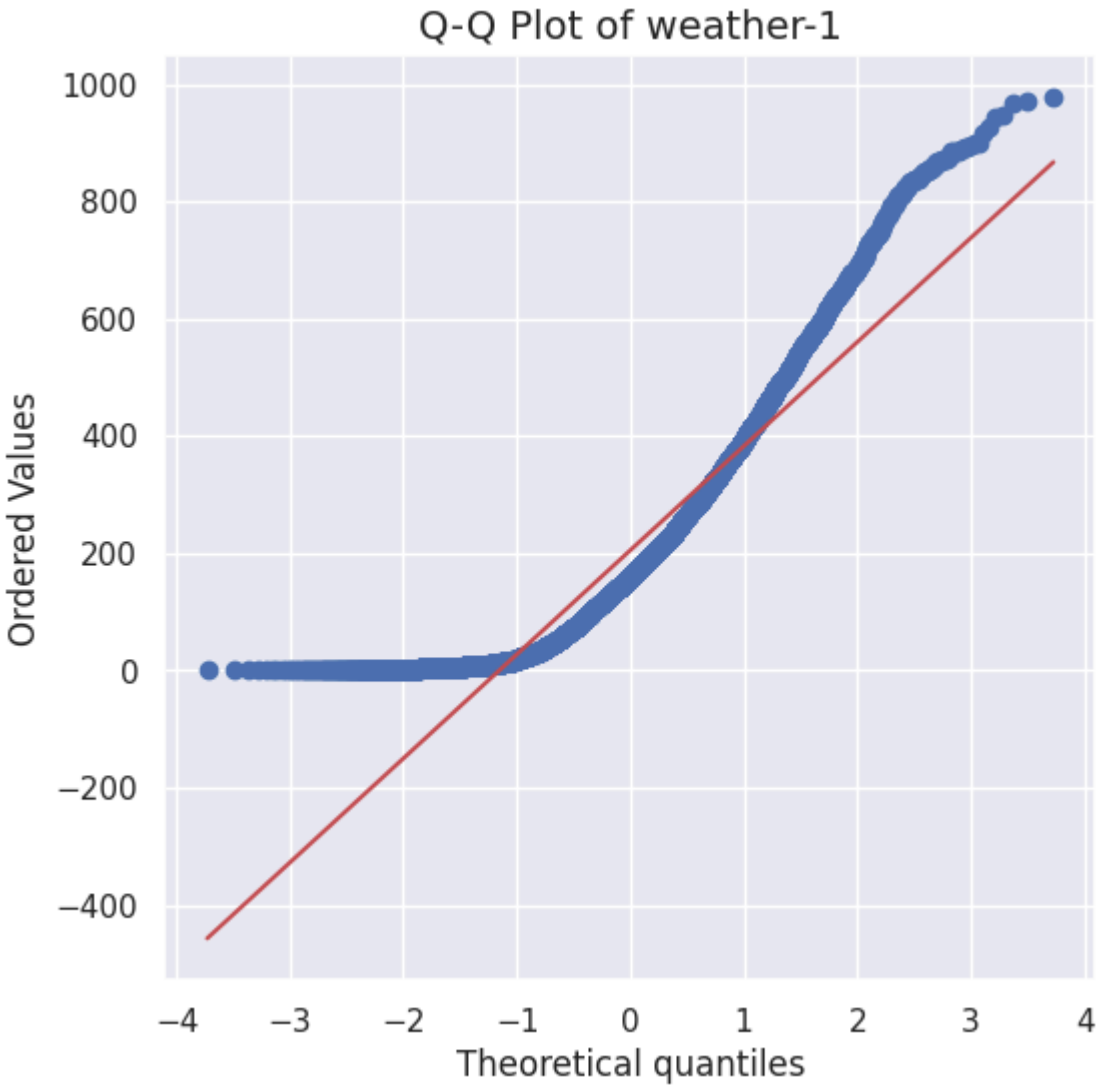

plot of weather-3
Count
250
200
150
100
50
0
0
200
400
600
800
count

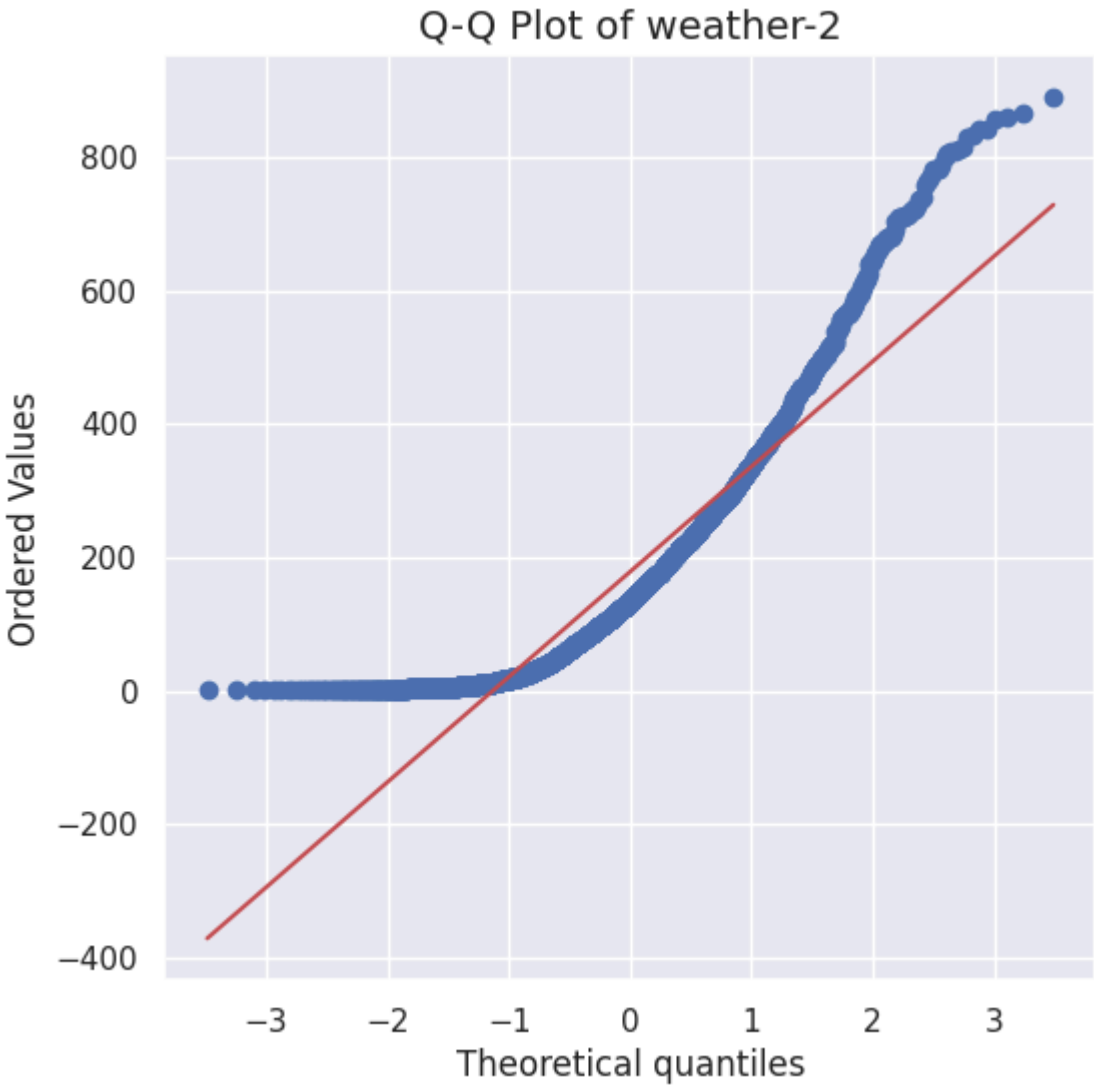


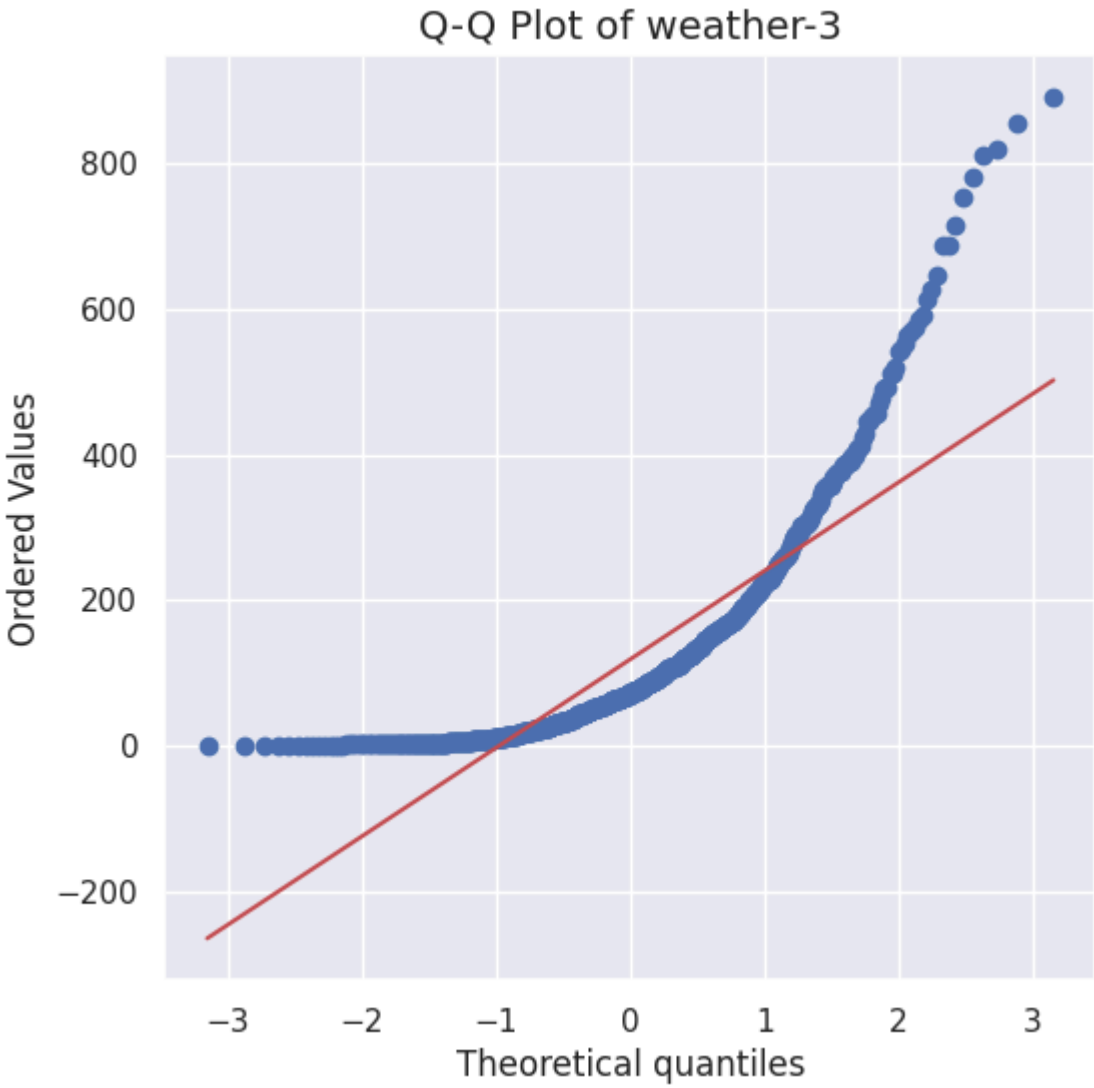
Q-Q plot for checking normality of data

In [35]:
```
from scipy import stats
```

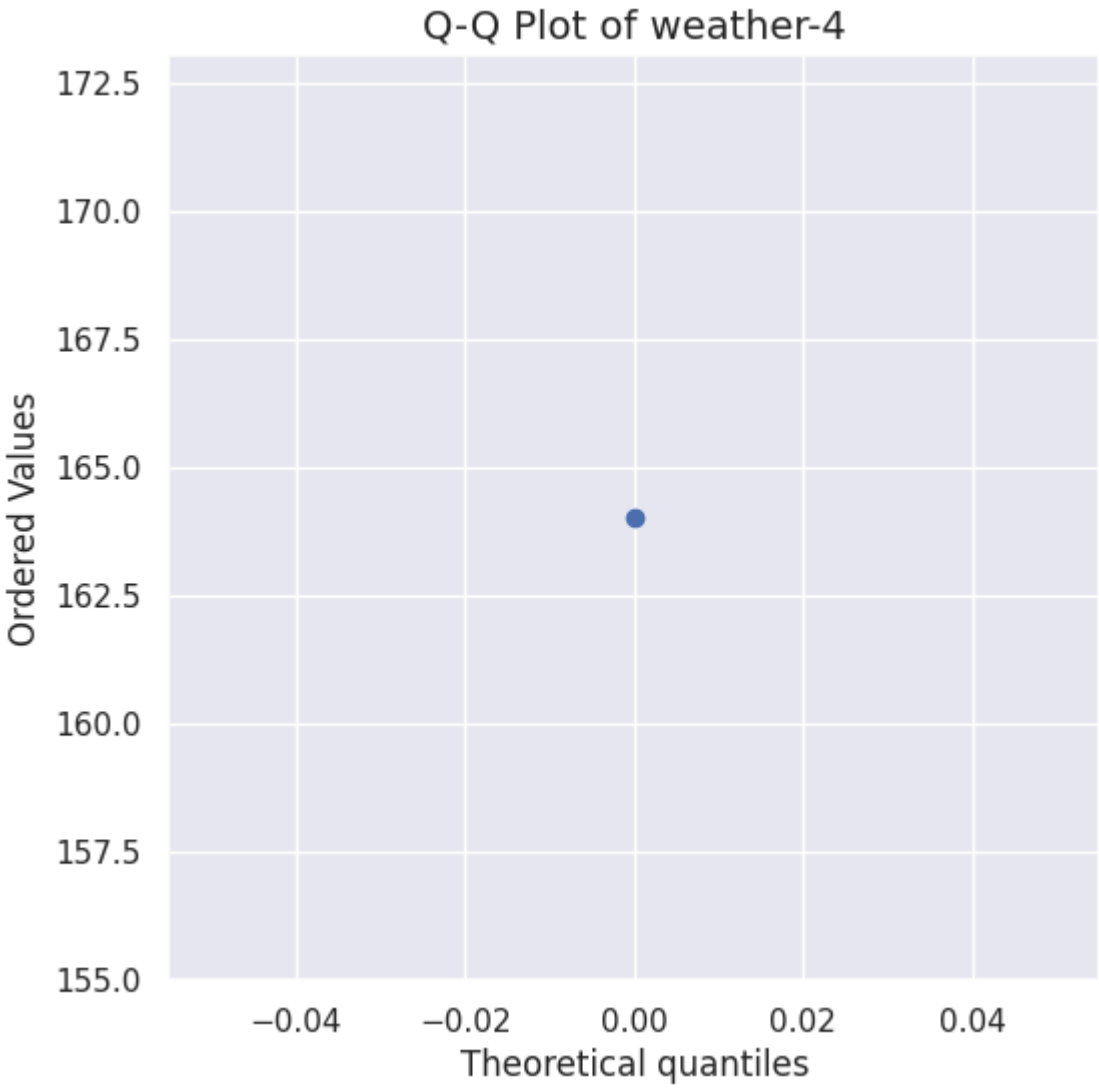
In [36]:
```
distributions = {
    'weather-1': dis1,
    'weather-2': dis2,
    'weather-3': dis3,
    'weather-4': dis4
}

for col_name, series in distributions.items():
    plt.figure(figsize=(6,6))
    stats.probplot(series, dist="norm", plot=plt)
    plt.title(f"Q-Q Plot of {col_name}", fontsize=14)
    plt.show()
```


Q-Q Plot of weather-1
Ordered Values
Theoretical quantiles
1000
800
600
400
200
0
−200
−400
−4
−3
−2
−1
0
1
2
3
4


Q-Q Plot of weather-2
Ordered Values
800
600
400
200
0
−200
−400
−3
−2
−1
0
1
2
3
Theoretical quantiles




```
/usr/local/lib/python3.12/dist-packages/scipy/stats/_morestats.py:640: Sma
llSampleWarning: One or more sample arguments is too small; all returned v
alues will be NaN. See documentation for sample size requirements.
  slope, intercept, r, prob, _ = _stats_py.linregress(osm, osr)
```



From these plots we can observe tha data is not normally distributed

Shapiro test for checking normality of data

In [37]:
```
from scipy.stats import shapiro
```

In [38]:
```
shapiro(dis1)
```

```
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:579: UserWarning: scipy.stats.shapiro: For N > 5000, computed p-value may not be accurate. Current N is 7192.
  res = hypotest_fun_out(*samples, **kwds)
```

Out[38]:
```
ShapiroResult(statistic=np.float64(0.8909259459740138), pvalue=np.float64(1.5964921477006555e-57))
```

In [39]:
```
shapiro(dis2)
```

Out[39]:
```
ShapiroResult(statistic=np.float64(0.8767694973495206), pvalue=np.float64(9.777839106111785e-43))
```

In [40]:
```
shapiro(dis3)
```

Out[40]:
```
ShapiroResult(statistic=np.float64(0.7674327906035717), pvalue=np.float64(3.875893017396149e-33))
```

2. levene's test for equality of variance

In [41]:
```
from scipy.stats import levene
```

In [42]:
```
levene(dis1,dis2,dis3,dis4)
```

Out[42]:
```
LeveneResult(statistic=np.float64(54.85106195954556), pvalue=np.float64
(3.504937946833238e-35))
```

All groups donot have equal variance

both assumptions are violated so we perfom kruskal wallis test

Kruskal wallis test

In [43]:
```
from scipy.stats import kruskal
```

Results

In [44]:
```
kruskal(dis1,dis2,dis3,dis4)
```

Out[44]:
```
KruskalResult(statistic=np.float64(205.00216514479087), pvalue=np.float6
4(3.501611300708679e-44))
```

pvalue<0.05 so we reject null hypothesis

Decision

The demand of bicycles on rent is not same for different Weather conditions

Checking if the demand of bicycles on rent is the same for different Seasons

Recommendation

Use weather forecast to proactively reallocate fleet (e.g., fewer bikes in heavy rain areas / offer promotions in rainy days).

Checking the demand of bicycles on rent is the same for different Seasons

Hypothesis

H0(Null Hypothesis):The demand of bicycles on rent is the same for different seasons

H1(Alternate Hypothesis):The demand of bicycles on rent is not same for different seasons

In [45]:
```
df['season'].value_counts()
```

Out[45]:

season	count
4	2734
2	2733
3	2733
1	2686

dtype: int64

In [46]:
```
df[['season','count']]
```

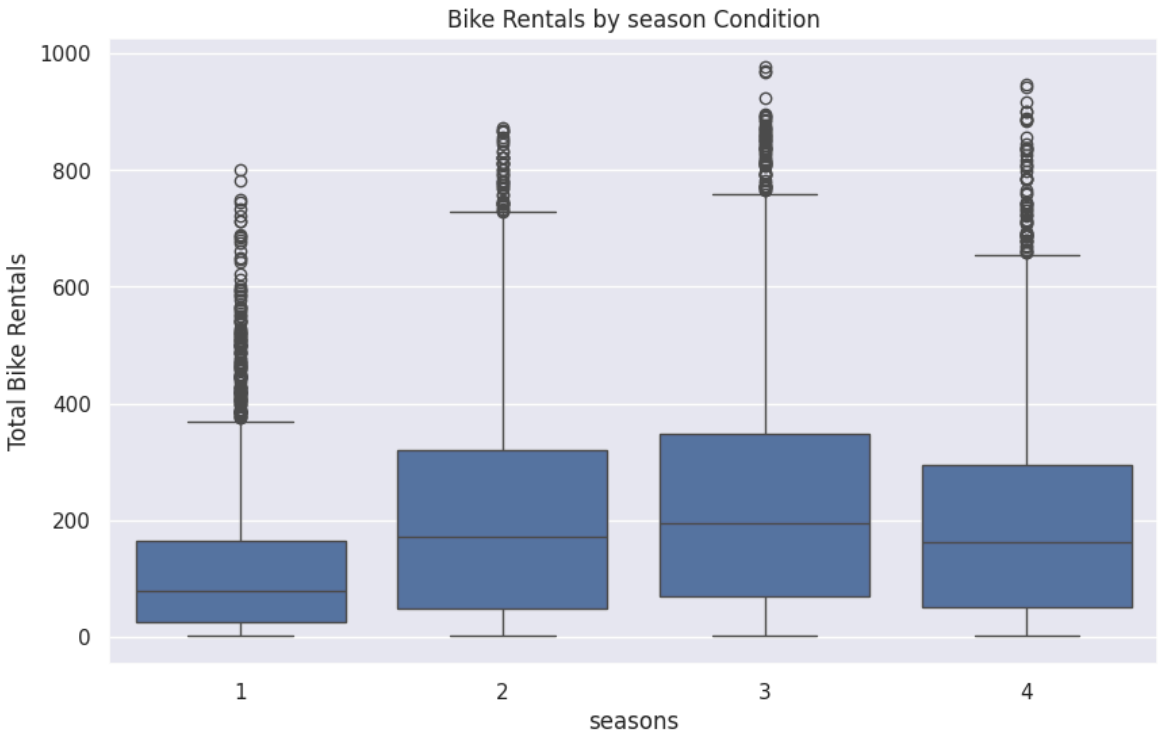
Out[46]:

	season	count
0	1	16
1	1	40
2	1	32
3	1	13
4	1	1
...	...	...
10881	4	336
10882	4	241
10883	4	168
10884	4	129
10885	4	88

10886 rows × 2 columns

First, let's observe the distribution of bike rentals across different seasons using a box plot. This will give us a visual idea of whether there are noticeable differences before performing the statistical test.

In [47]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(x='season', y='count', data=df)
plt.title('Bike Rentals by season Condition')
plt.xlabel('seasons')
plt.ylabel('Total Bike Rentals')
plt.show()
```



Tests

When we have one categorical variable with more than two groups and one continuous variable we consider one way anova test if assumptions are met and if not met we perform kruskal wallis test

Assumptions of anova

1. Normality of data we use shapiro wilk test for checking normality of data

In [48]:
```
disseason1=df[df['season']==1]['count']
```

In [49]:
```
disseason2=df[df['season']==2]['count']
```

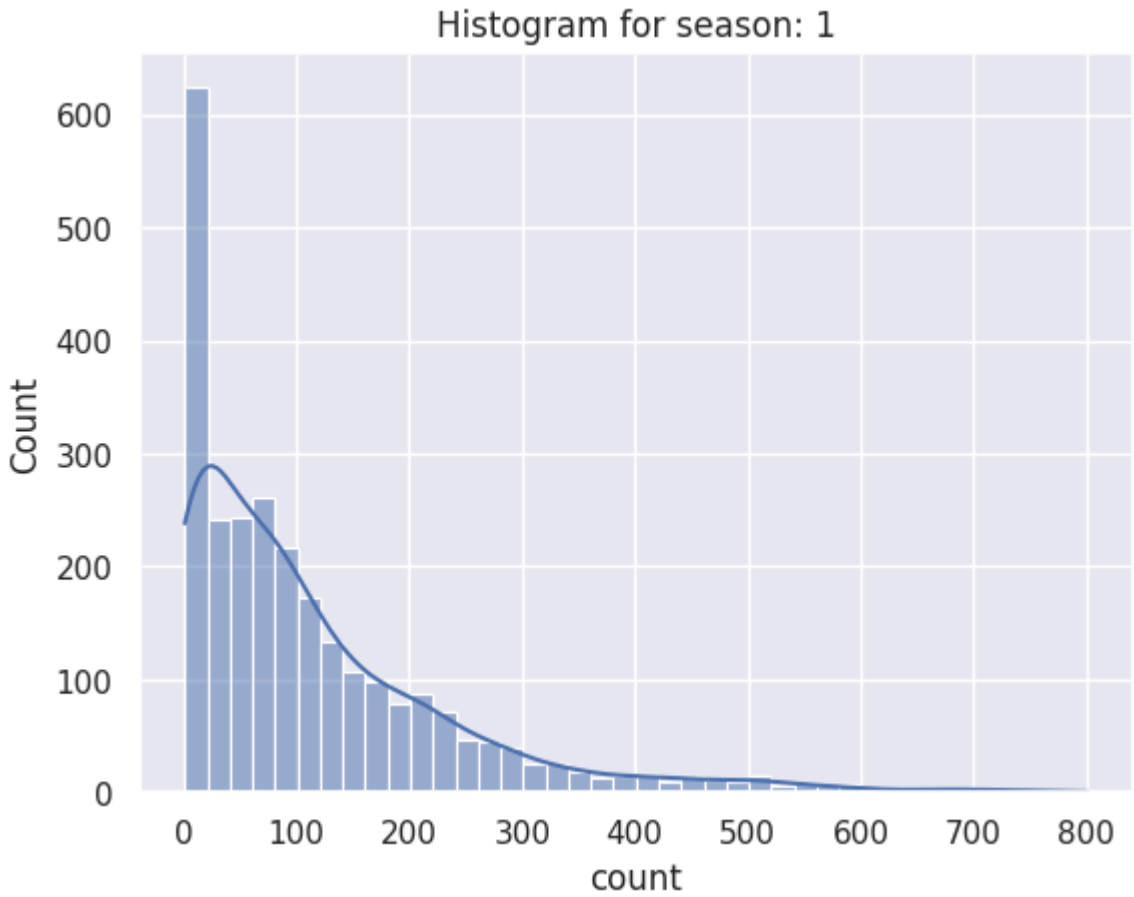
In [50]:
```
disseason3=df[df['season']==3]['count']
```

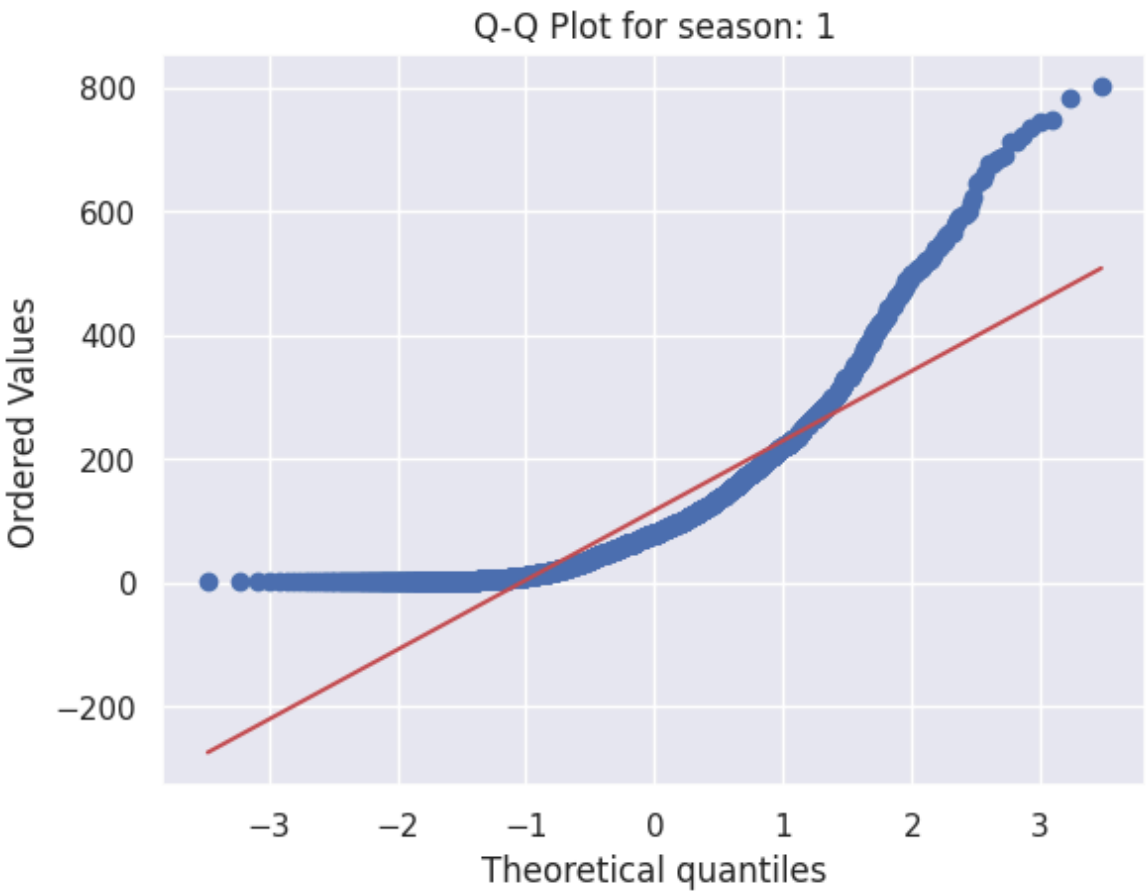
In [51]:
```
disseason4=df[df['season']==4]['count']
```

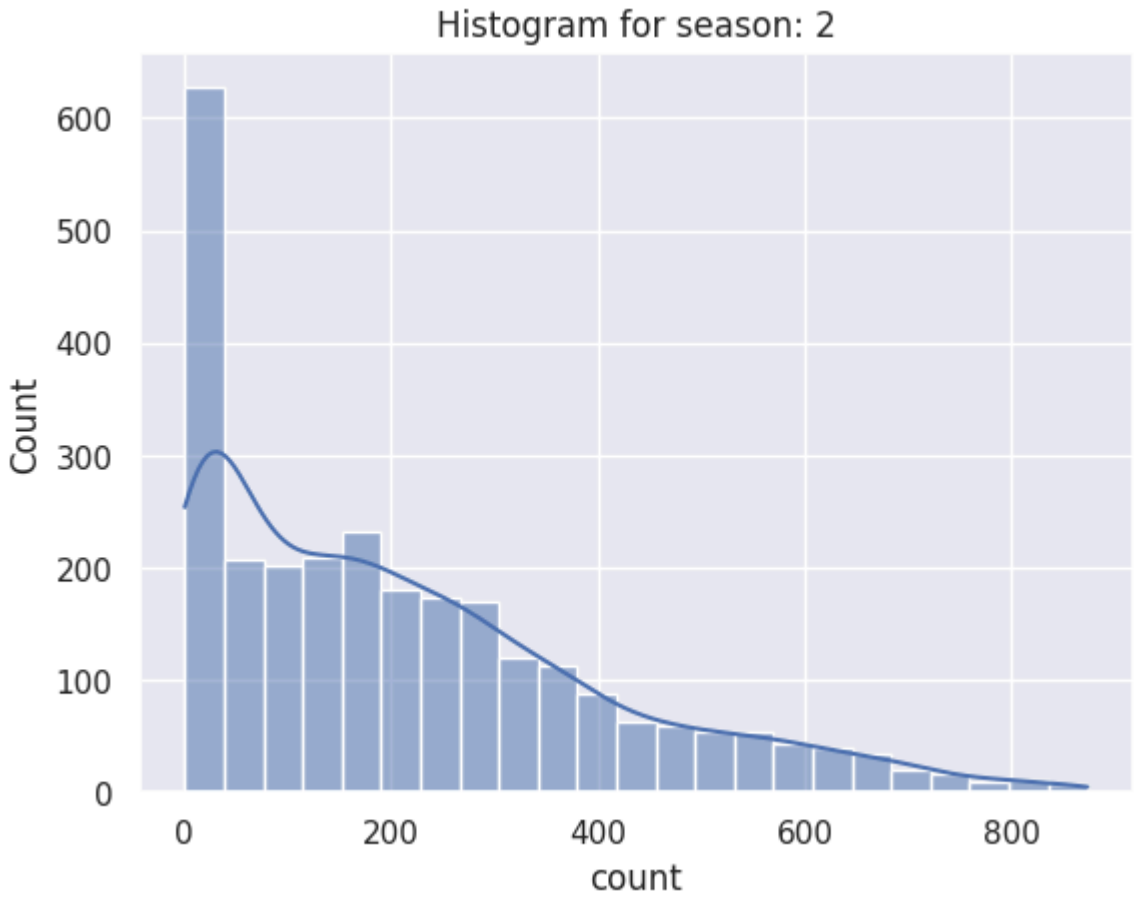
In [52]:
```
df['season'].unique()
```

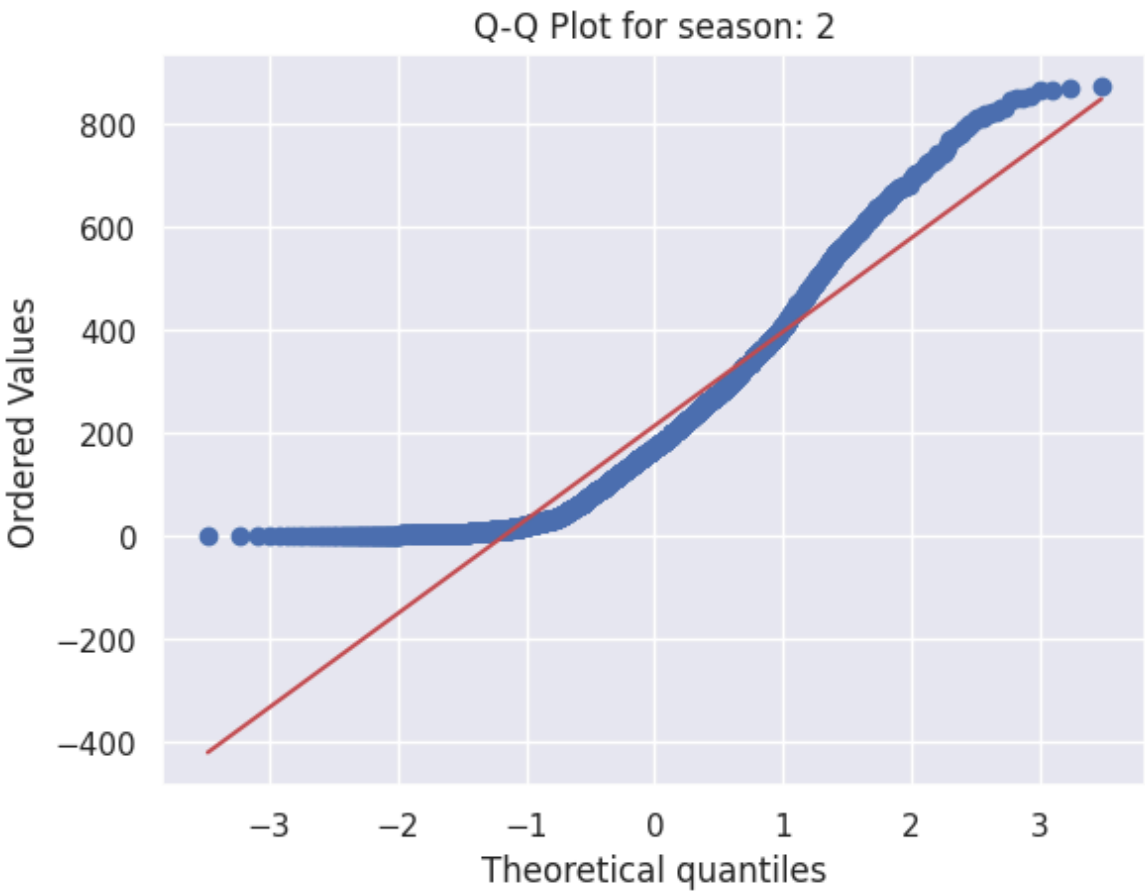
Out[52]:
```
array([1, 2, 3, 4])
```

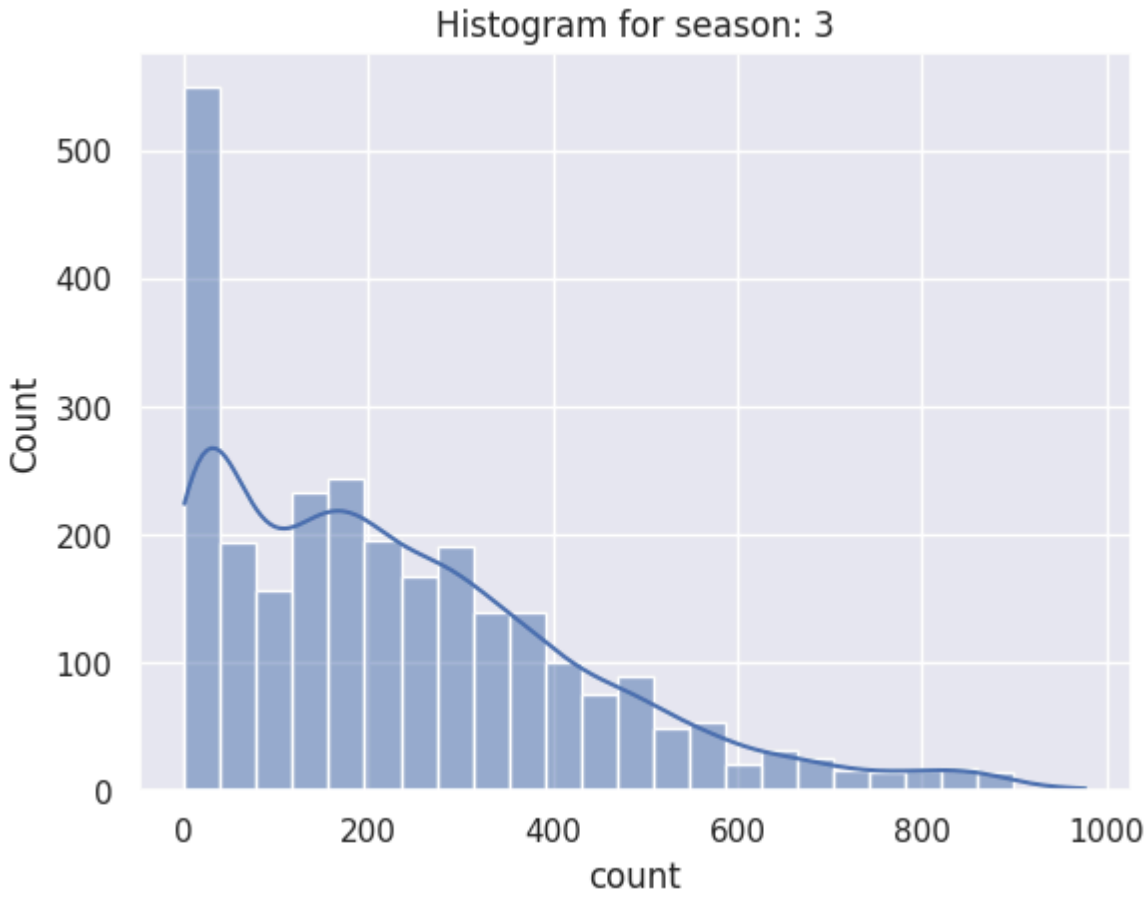
In [53]:
```
for w in df['season'].unique():
    sns.histplot(df[df['season'] == w]['count'], kde=True)
    plt.title(f"Histogram for season: {w}")
    plt.show()

    stats.probplot(df[df['season'] == w]['count'], dist="norm", plot=plt)
    plt.title(f"Q-Q Plot for season: {w}")
    plt.show()
```

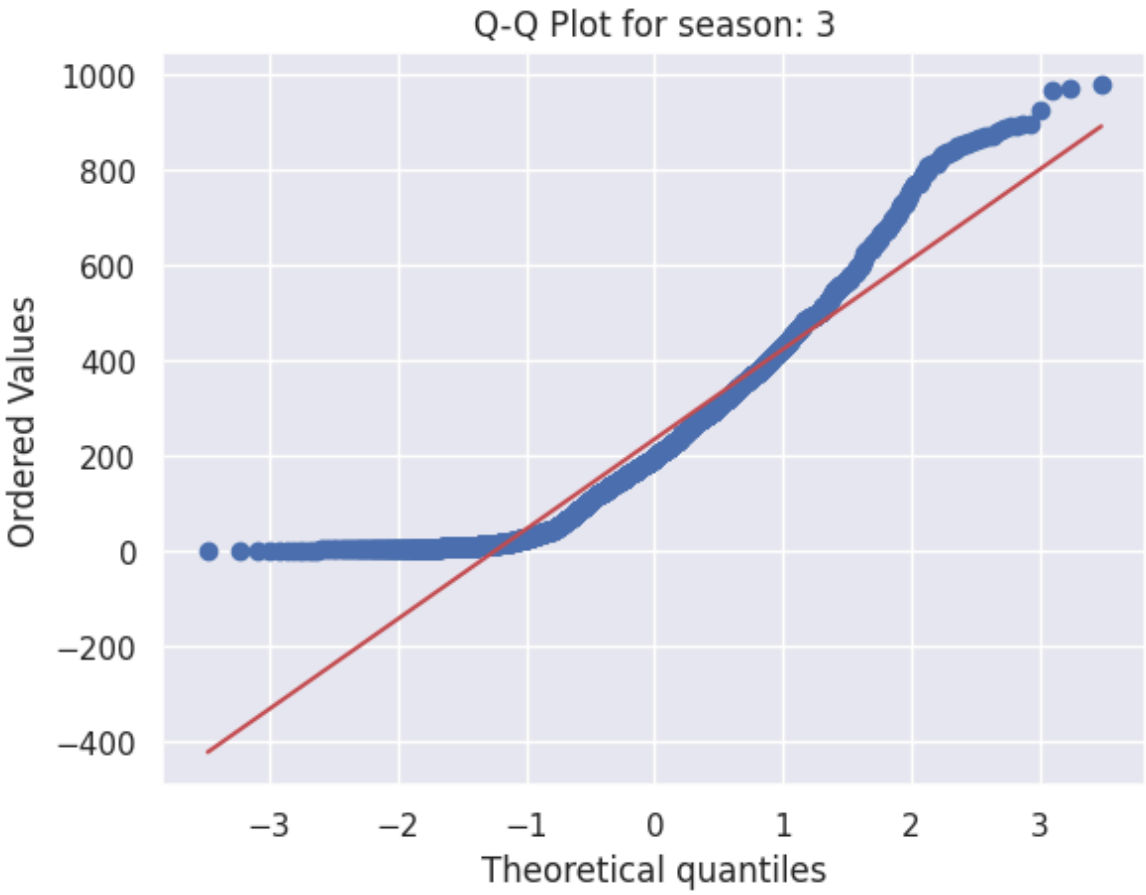

Histogram for season: 1
Count
count
0
100
200
300
400
500
600
700
800

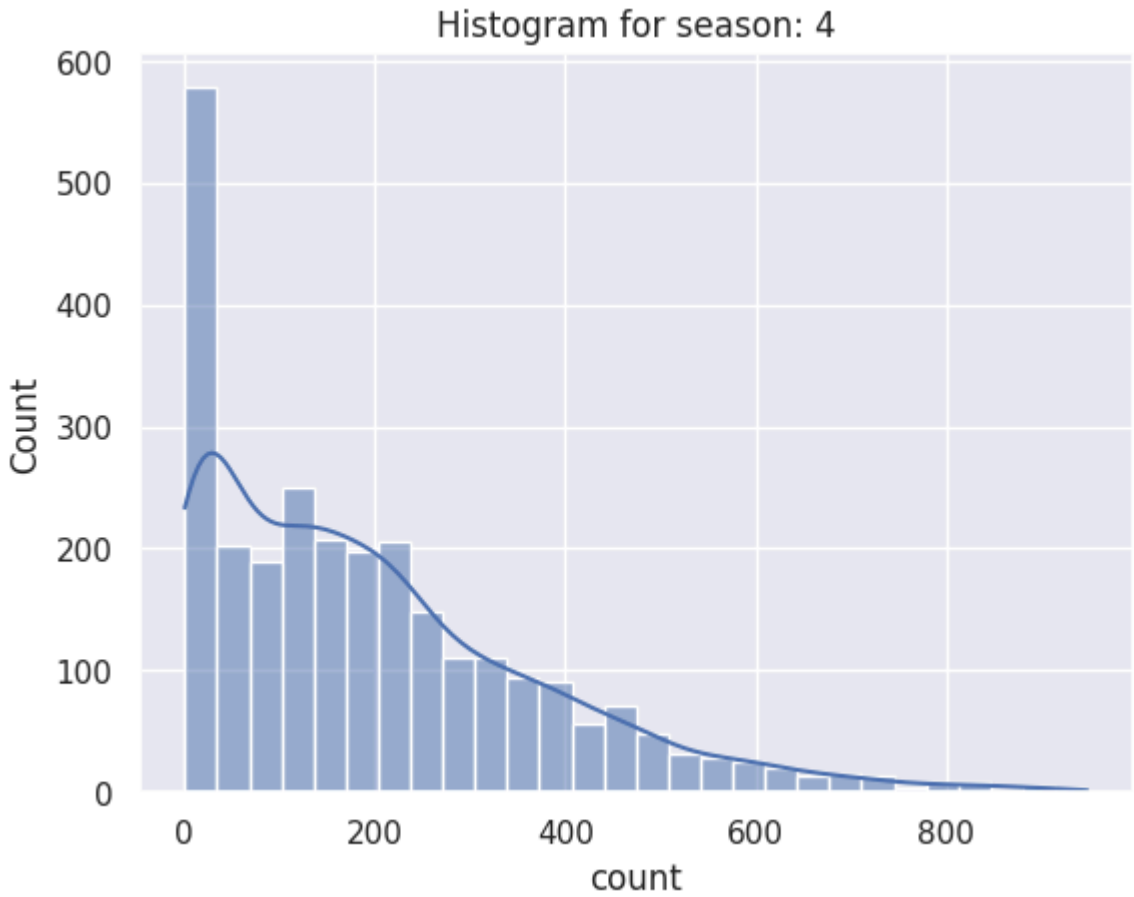

Q-Q Plot for season: 1
Ordered Values
Theoretical quantiles
−200
0
200
400
600
800
−3
−2
−1
0
1
2
3

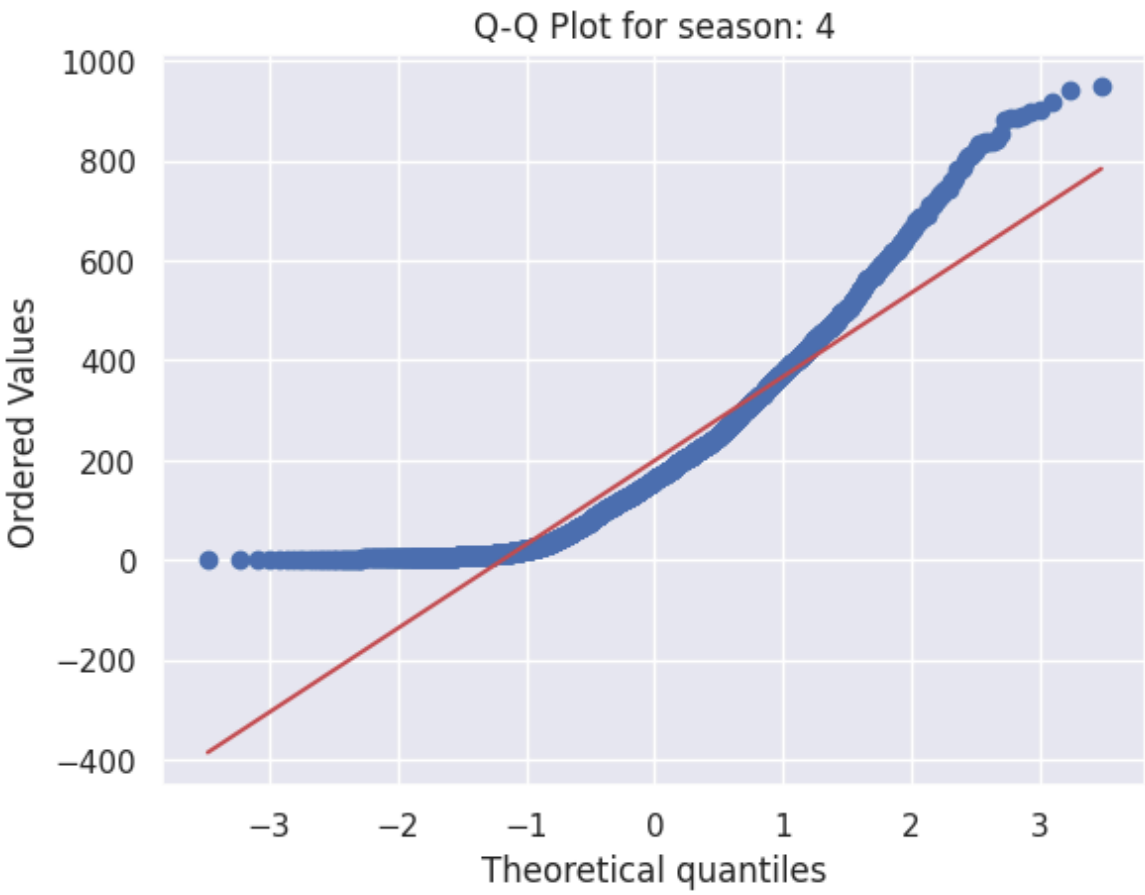

Histogram for season: 2
Count
count
0
100
200
300
400
500
600
0
200
400
600
800


Q-Q Plot for season: 2
Ordered Values
Theoretical quantiles
−400
−200
0
200
400
600
800
−3
−2
−1
0
1
2
3


Histogram for season: 3
Count
500
400
300
200
100
0
0
200
400
600
800
1000
count


Q-Q Plot for season: 3
Ordered Values
1000
800
600
400
200
0
−200
−400
−3
−2
−1
0
1
2
3
Theoretical quantiles





From these Q-Q plots we can observe that data is not normally distributed

Shapiro test for Normality of data

In [54]:
```
shapiro(disseason1)
```

Out[54]:
```
ShapiroResult(statistic=np.float64(0.8087378401253588), pvalue=np.float6
4(8.749584618867662e-49))
```

In [55]:
```
shapiro(disseason2)
```

Out[55]:
```
ShapiroResult(statistic=np.float64(0.9004818080893252), pvalue=np.float6
4(6.039374406270491e-39))
```

In [56]:
```
shapiro(disseason3)
```

Out[56]:
```
ShapiroResult(statistic=np.float64(0.9148166372899196), pvalue=np.float6
4(1.043680518918597e-36))
```

In [57]:
```
shapiro(disseason4)
```

Out[57]:
```
ShapiroResult(statistic=np.float64(0.8954637482095505), pvalue=np.float6
4(1.1299244409282836e-39))
```

2)levene's test for equality of variance

In [58]:
```
levene(disseason1,disseason2,disseason3,disseason4)
```

Out[58]:
```
LeveneResult(statistic=np.float64(187.7706624026276), pvalue=np.float64
(1.0147116860043298e-118))
```

Atleast one group has different variance

both assumptions are violated so we perfom kruskal wallis test

results

In [59]:
```
kruskal(disseason1,disseason2,disseason3,disseason4)
```

Out[59]:
```
KruskalResult(statistic=np.float64(699.6668548181988), pvalue=np.float64
(2.479008372608633e-151))
```

Decision

pvalue<0.05 so we reject null hypothesis

the demand of bicycles on rent is the different for different Seasons

Recommendations

Seasonal fleet planning: scale up in high-demand seasons, schedule maintenance in off-season.

Use season as a key variable in predictive models.

Checking if the Weather conditions are significantly different during different Seasons

Hypothesis

H0(Null Hypothesis):the Weather conditions are not significantly different during different Seasons H1(Alternate Hypothesis):the Weather conditions are significantly different during different Seasons

Test

for two categorical columns we use chi square test

In [60]:
```
df[['weather','season']]
```

Out[60]:

	weather	season
0	1	1
1	1	1
2	1	1
3	1	1
4	1	1
...	...	...
10881	1	4
10882	1	4
10883	1	4
10884	1	4
10885	1	4

10886 rows × 2 columns

Contingency Table

First, a contingency table was created to cross-tabulate the 'weather' and 'season' columns. This table shows the frequency of each weather condition occurring within each season.

In [61]:
```
table=pd.crosstab(df['weather'],df['season'])
table
```

Out[61]:

season	1	2	3	4
weather				
1	1759	1801	1930	1702
2	715	708	604	807
3	211	224	199	225
4	1	0	0	0

Chi-square Test

Next, the Chi-square test for independence was performed on the contingency table to determine if there is a statistically significant relationship between weather conditions and seasons.

In [62]:
```
from scipy.stats import chi2_contingency
```

In [63]:
```
chi2_contingency(table)
```

Out[63]:
```
Chi2ContingencyResult(statistic=np.float64(49.15865559689363), pvalue=n
p.float64(1.5499250736864862e-07), dof=9, expected_freq=array([[1.774546
39e+03, 1.80559765e+03, 1.80559765e+03, 1.80625831e+03],
       [6.99258130e+02, 7.11493845e+02, 7.11493845e+02, 7.11754180e+02],
       [2.11948742e+02, 2.15657450e+02, 2.15657450e+02, 2.15736359e+02],
       [2.46738931e-01, 2.51056403e-01, 2.51056403e-01, 2.51148264e-0
1]]))
```

Results

In [68]:
```
chi,p,dof,expected=chi2_contingency(table)
print(f"Chi-square statistic: {chi:.2f}")
print(f"P-value: {p:.10f}")
print(f"Degrees of freedom: {dof}")
```

```
Chi-square statistic: 49.16
P-value: 0.0000001550
Degrees of freedom: 9
```

Decision

pvalue<0.05 which indicates rejection of null hypothesis

the Weather conditions are significantly different during different Seasons

The formulated hypotheses provide a clear framework for conducting the Chi-square test to statistically evaluate the relationship between weather conditions and seasons.

Recommendation

Use season to forecast likely weather distribution; rely on season + forecast in operational decisions.